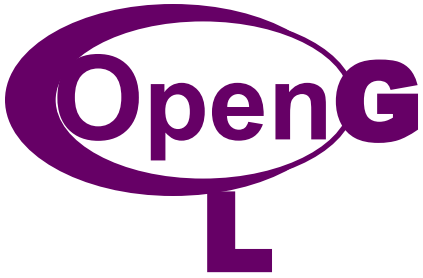
Department of Computer Science

Computer Graphics and Visualization

Unit 2



By

Dr. Srividya M S

Objectives

- Development of the OpenGL API
- OpenGL as state machine
- Fundamental OpenGL primitives
- Attributes
- Simple viewing
  - Two-dimensional viewing as a special case of 3D viewing
- Interaction with the Window System
- Sierpinski Gasket Program – triangles
- 3D Sierpinski Gasket Program
- Introduce hidden-surface removal

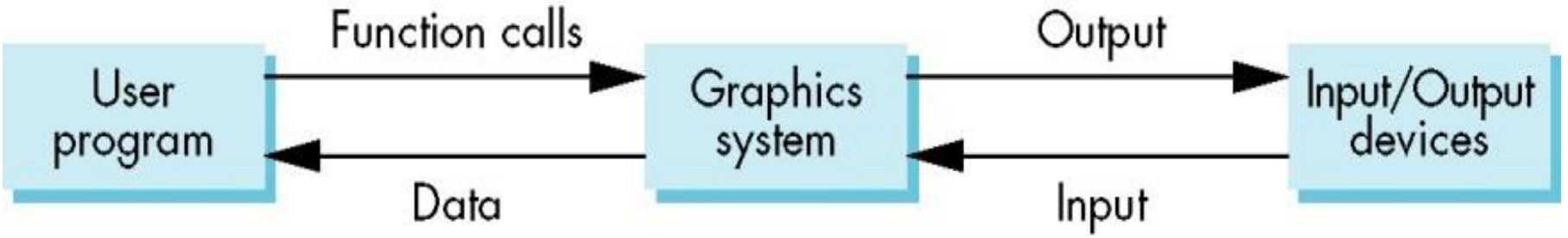
Why OpenGL API ?

- OpenGL is not a programming language like C or C++. It is more like ***C runtime library*** which provides some prepackaged functionality.
- Device independent, platform independent API.
- Defined as a ***software interface to graphics hardware***.
- 3D graphics and modeling library that is extremely portable and very fast.
- Using OpenGL, elegant 3D graphics can be created.
- It uses algorithms developed and optimized by Silicon Graphics Inc(SGI) , an acknowledged world leader in graphics and animation.

Graphics functions

Graphics functions

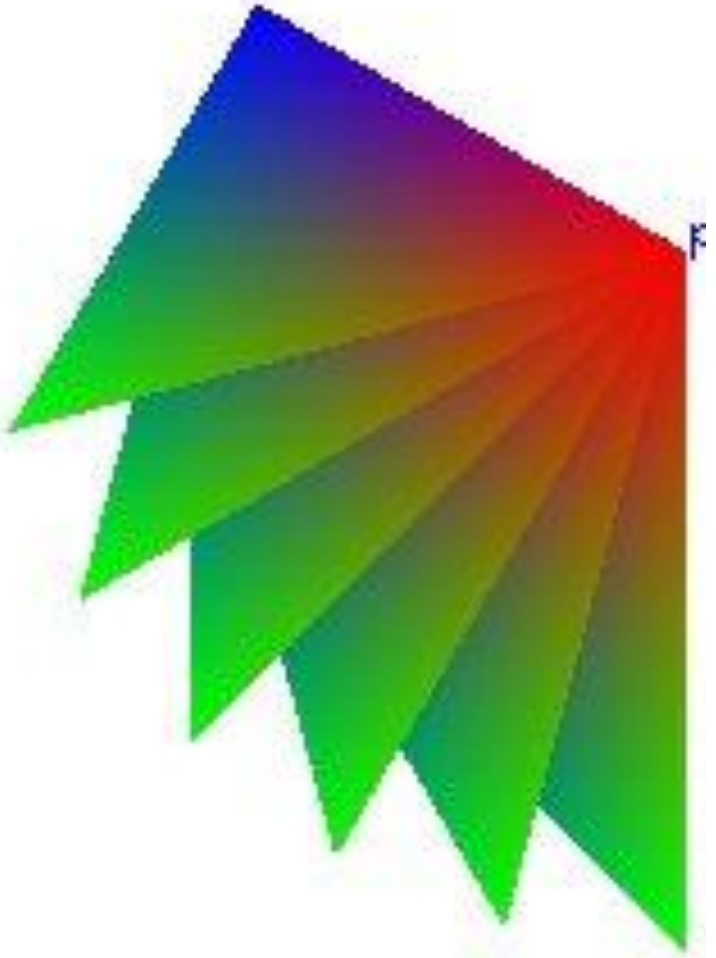
- Basic model of graphics package is ***a black box.***
- ***Black box*** - a term that engineers use to denote a system whose properties are described by only its inputs and outputs; nothing is known about its internal workings.



Graphics functions

- Hence graphics system can be viewed as a box whose
  - ***inputs*** are ***function calls*** from a user program (say measurements from input devices, such as the mouse and keyboard and possibly other input, such as messages from the operating system).
  - ***outputs*** are primarily the ***graphics*** sent to the output devices.

Graphics Functions - Types



Graphics Functions

Graphics functions can be grouped as

1. Primitive functions
2. Attribute functions
3. Viewing functions
4. Transformation functions
5. Input functions
6. Control functions
7. Query functions

1. Primitive functions
2. Attribute functions
3. Viewing functions
4. Transformation functions
5. Input functions
6. Control functions
7. Query functions

Primitive functions enable the programmer to create some ***basic primitives*** such as points, line segments, polygons, text etc.

Example :

glVertex*();

glBegin();

glEnd();

1. Primitive functions
2. Attribute functions
3. Viewing functions
4. Transformation functions
5. Input functions
6. Control functions
7. Query functions

Example :

```
/* Draw a triangle */

glBegin (GL_LINE_LOOP);
    glVertex3f(-0.3, -0.3, 0.0);
    glVertex3f(0.0, 0.3, 0.0);
    glVertex3f(0.3, -0.3, 0.0);
glEnd();
```

1. Primitive functions
2. Attribute functions
3. Viewing functions
4. Transformation functions
5. Input functions
6. Control functions
7. Query functions

Attribute functions perform operations ranging from ***choosing the color***, to ***picking a pattern*** with which to fill the inside of a polygon, to selecting a type face for the titles on a graph.

Example :

```
glColor3f(1.0,0.0,0.0);
glClearColor(1.0,1.0,1.0,1.0);
```

1. **Primitive functions**
2. **Attribute functions**
3. **Viewing functions**
4. **Transformation functions**
5. **Input functions**
6. **Control functions**
7. **Query functions**

Viewing functions enable the programmer to specify ***views*** such as front view, top view, side view. Also provide ***zoom in*** and ***zoom out*** facilities.

Example :

```
glViewport();
glMatrixMode(GL_PROJECTION);
glMatrixMode(GL_MODELVIEW);
gluOrtho2D();
```

1. Primitive functions
2. Attribute functions
3. Viewing functions
4. Transformation functions
5. Input functions
6. Control functions
7. Query functions

Transformation Functions enable the programmer to carryout geometric transformations such as ***rotation, scaling, translation***.

Example :

glTranslatef();

glRotatef();

1. Primitive functions
2. Attribute functions
3. Viewing functions
4. Transformation functions
5. Input functions
6. Control functions
7. Query functions

Input functions enable the programmer to design interactive programs.

These functions can accept ***inputs from mouse, keyboard, data tablets*** etc.

Example :

```
glutMouseFunc(mouse);
```

1. **Primitive functions**
2. **Attribute functions**
3. **Viewing functions**
4. **Transformation functions**
5. **Input functions**
6. **Control functions**
7. **Query functions**

Control functions enable the programmer to ***communicate*** with the window system, initialize programs, deal with errors that occur during the execution of the program etc.

Example :

```
glutInitDisplayMode();
glutInitWindowSize();
glutInitWindowPosition();
```

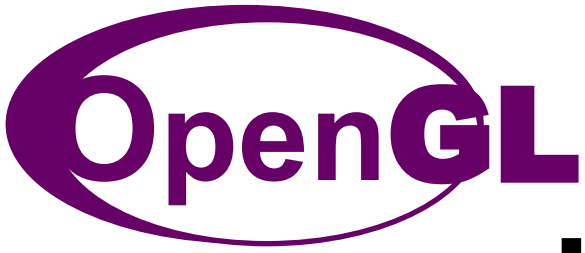
1. **Primitive functions**
2. **Attribute functions**
3. **Viewing functions**
4. **Transformation functions**
5. **Input functions**
6. **Control functions**
7. **Query functions**

Query functions enable the programmer to access information such as ***camera parameters***, contents of the frame buffer etc which the programmer can design device independent programs.

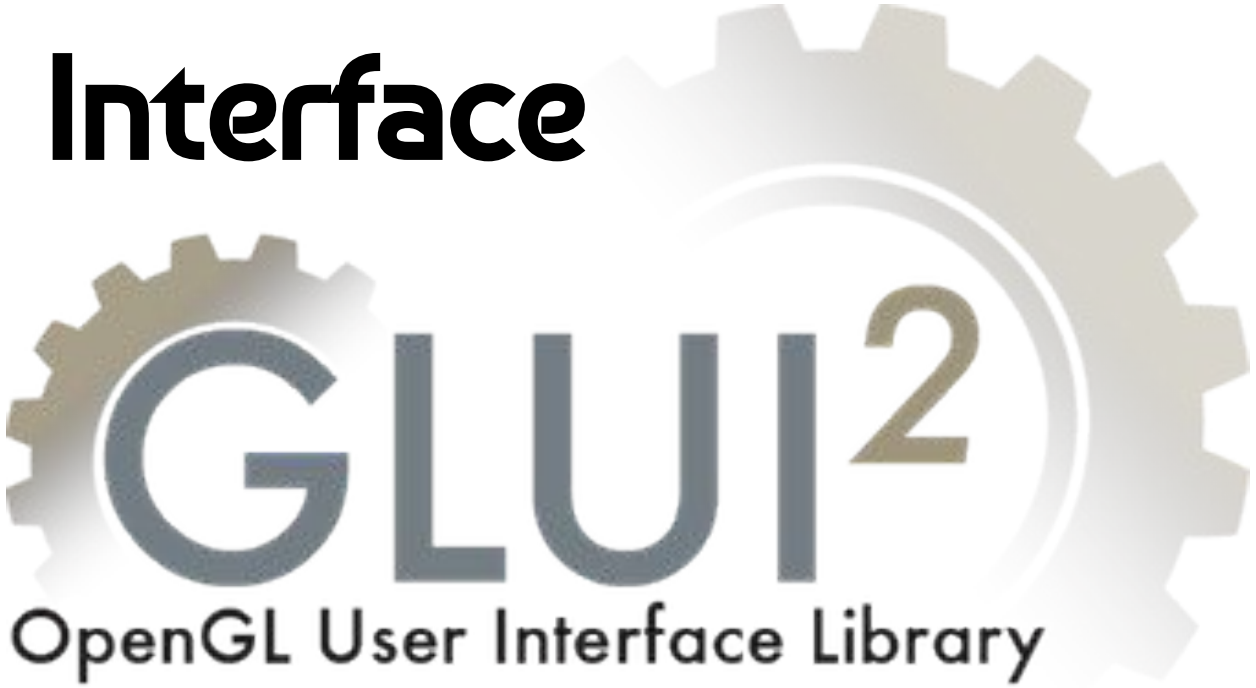
Example :

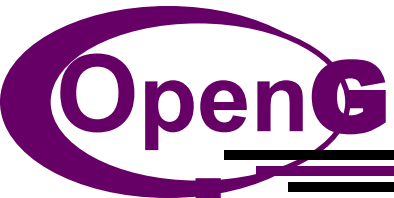
glGetBooleanV();

glGetIntegerV();



Interface





The OpenGL Interface

The three main libraries of OpenGL Interface

1. **Graphics Library (GL)**
2. **Graphics Library Utility (GLU)**
3. **GL Utility Toolkit (GLUT)**

The OpenGL Interface

1. Graphics Library (GL):

- Supports creation of a small set of primitives.
- OpenGL function names begin with the letters **gl.**
- They are stored in a library referred to as *GL (OpenGL in windows).*

The OpenGL Interface

2. Graphics Utility Library (GLU)

- Supports creation of a large set of primitives.
- This library uses only GL functions.
- But it contains code for common objects, such as spheres.
- This library is available in all OpenGL implementations. Functions in the GLU library begin with **glu.**

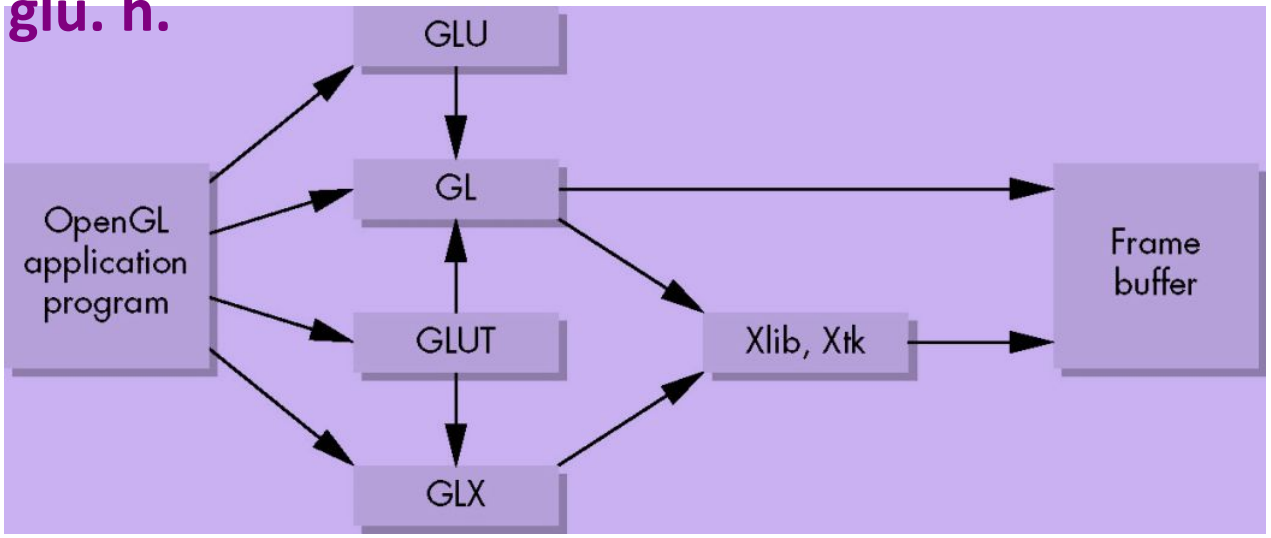
The OpenGL Interface

3. GL Utility Toolkit (GLUT)

- To interface with the window system and to get input from external devices into the program, one more library is needed.
  - X Windows System - GLX
  - Windows – wgl
  - Macintosh – agl
- But instead of using a diff library every time, we use readily available GLUT – provides min functionality expected in any modern windowing system.

The OpenGL Interface

- Figure below shows the organization of the libraries for an **X Window system** environment.
- Other libraries are called from the OpenGL libraries, but that the application program does not need to refer to these libraries directly.
- In most implementations, one of the include lines **#include <GL/glut.h>** or **#include <glut.h>** is sufficient to read in **glut.h, gl.h**, and **glu. h.**



OpenGL Primitives & Attributes

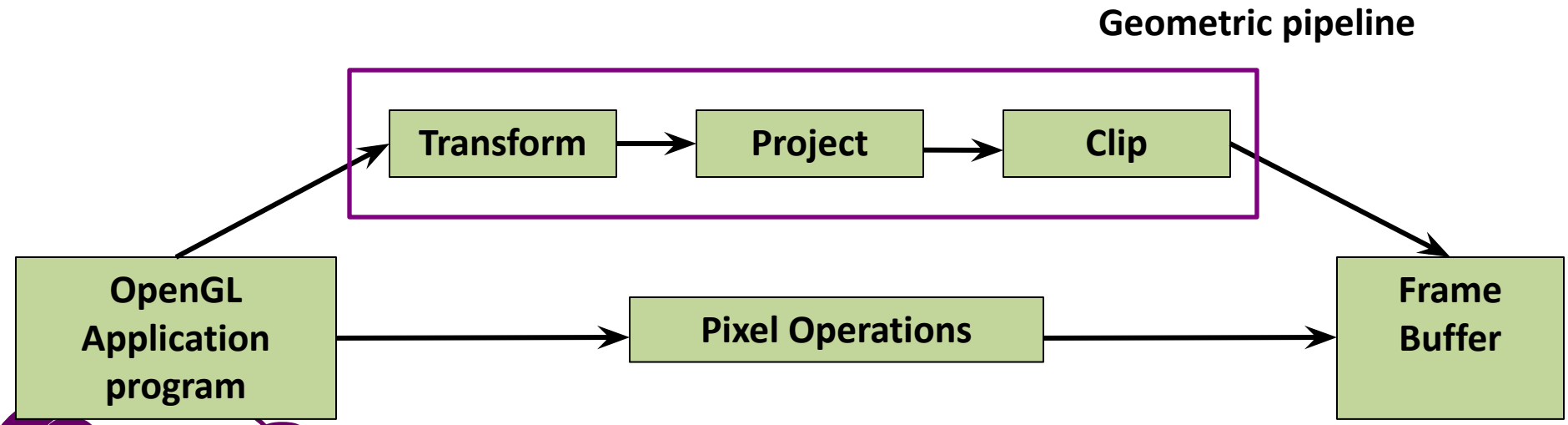
OpenGL supports the following primitives

- **Geometric primitives**

  Points, line segments, polygons, curves, surfaces

- **Image OR raster primitives**

  Array of pixels



Geometric primitives

Geometric primitives syntax :

```
glBegin(type);
    glVertex*(...);
    ..........
    glVertex*(...);
glEnd ( );
```

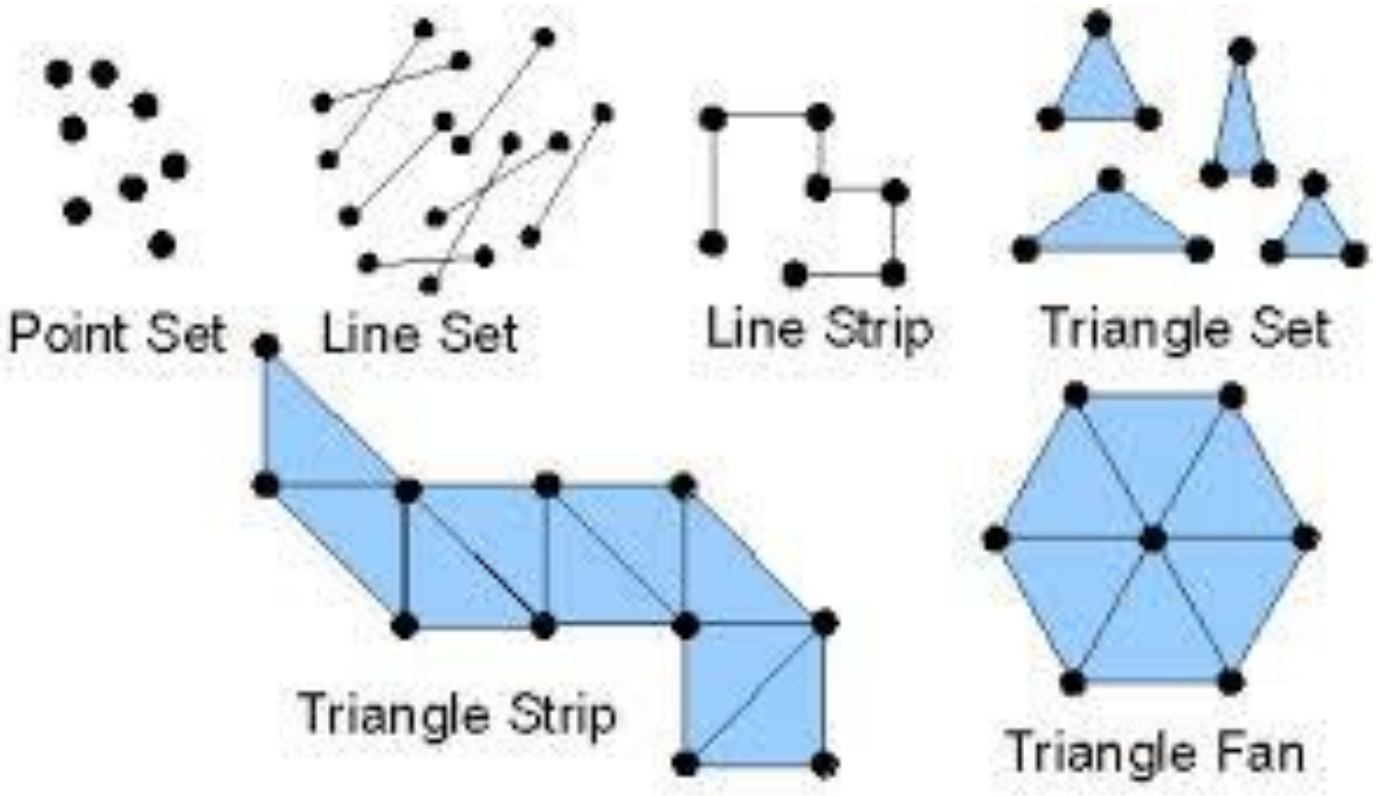
type specifies how openGL assembles the vertices to define geometric objects.

Geometric primitives

The geometric primitives and their type specification as follows

1. **Points : (GL_POINTS)**
2. **Line segment : (GL_LINES)**
3. **Polylines : (GL_LINE_STRIP, GL_LINE_LOOP)**

OpenGL Primitives & Attributes



Programming 2-D Applications

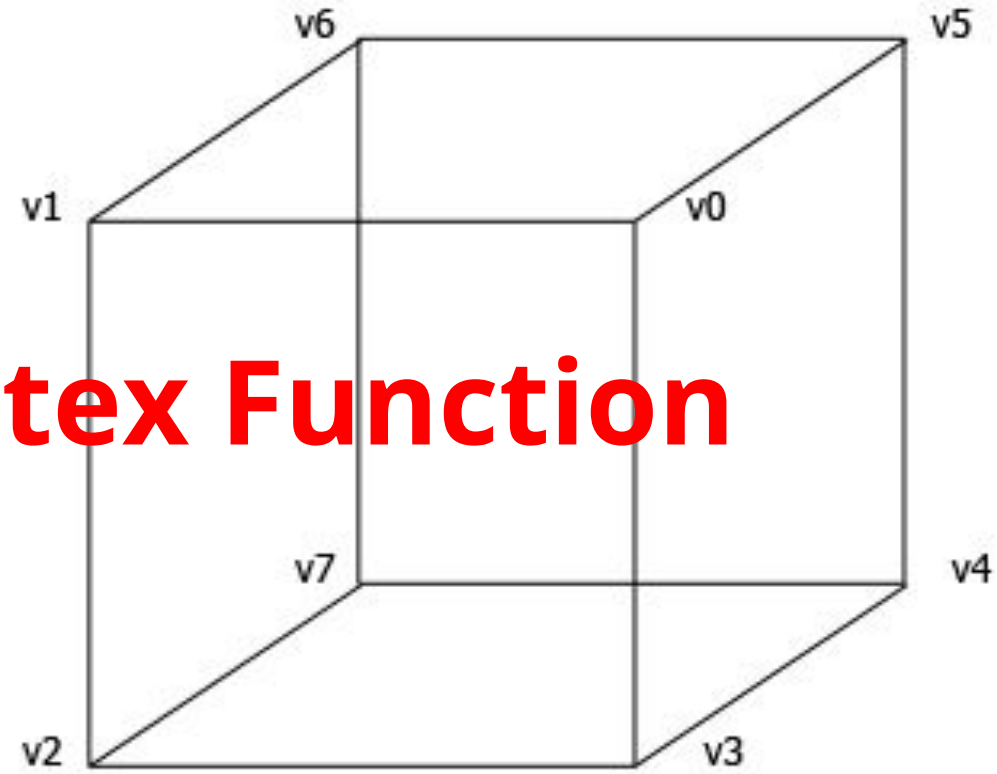
- In OpenGL, internal representation for both 2-D and 3-D points is same. Hence a 3-D point is represented by a triplet:

$$P = \begin{pmatrix} x \\ y \\ z \end{pmatrix}$$

- ***Vertex (* rather than *point):***
  - A vertex is a location in space (in Computer Graphics a 2-D, 3-D, 4-D spaces are used).
  - Vertices are used to define the atomic geometric objects that are recognized by graphics system.

Programming 2-D Applications

- The simplest geometric object is a point in space, which is specified by a single vertex.
- Two vertices define a line segment.
- Three vertices can determine either a triangle or a circle.
- Four vertices determine a quadrilateral, and so on.



Vertex Function

Vertex Function

- A vertex is a position in space.
- Space can be 2-D or 3-D.
- Vertex is used to define geometric primitives.

Example

Two vertices define a line segment. In OpenGL it is represented as **glVertex*()** where * can be nt OR ntv

n no. of dimensions (2, 3 , 4)

t data types (int, float, double)

v specifies that variables are specified through a pointer to an array.

Basic OpenGL data types include

GLfloat, Glint, GLdouble

Vertex Function

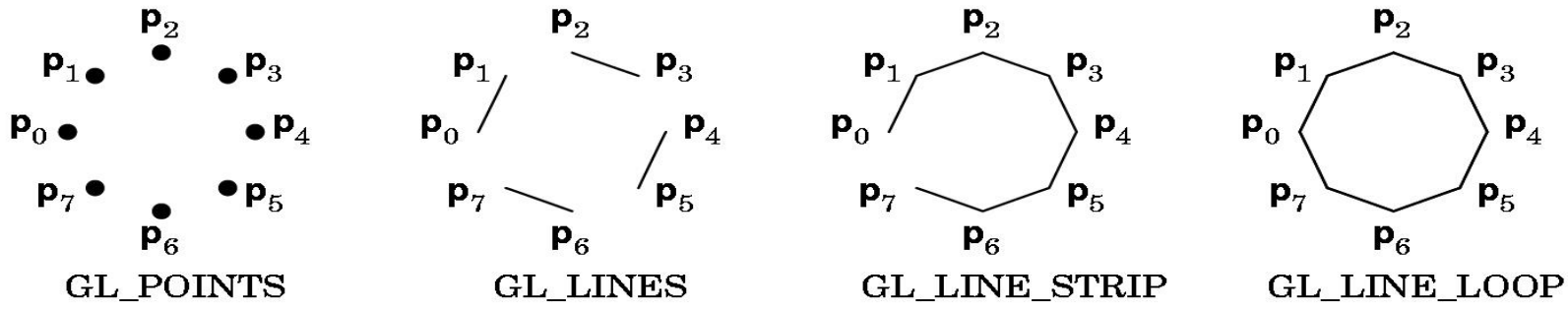
Vertex in 2D can be represented as
glVertex2i(GLint xi, GLint yi)

In OpenGL, primitives are defined between ***glBegin*** and ***glEnd***

glBegin – The argument specifies the geometric type of vertices.

Example :

```
glBegin(GL_LINES);
    glVertex3f(x1, y1, z1);
    glVertex3f(x2, y2, z2);
glEnd();
```



GL_POINTS : Each vertex is displayed at a size of at least one pixel.

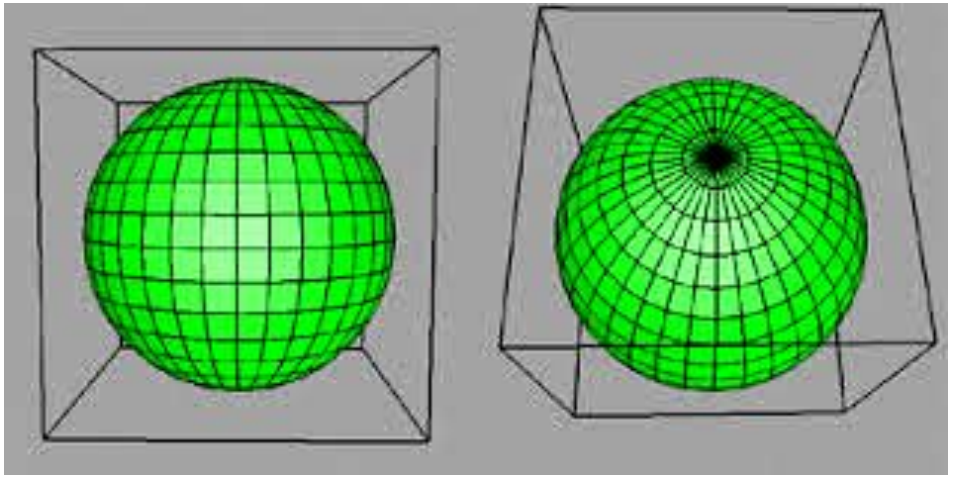
GL_LINES : Successive pairs of vertices would be connected as a line.

GL_LINE_STRIP : Connects the successive vertices using line segments. However the final vertex would not be connected to the initial vertex.

GL_LINE_LOOP : Connects the successive vertices using line segments to form a closed path.

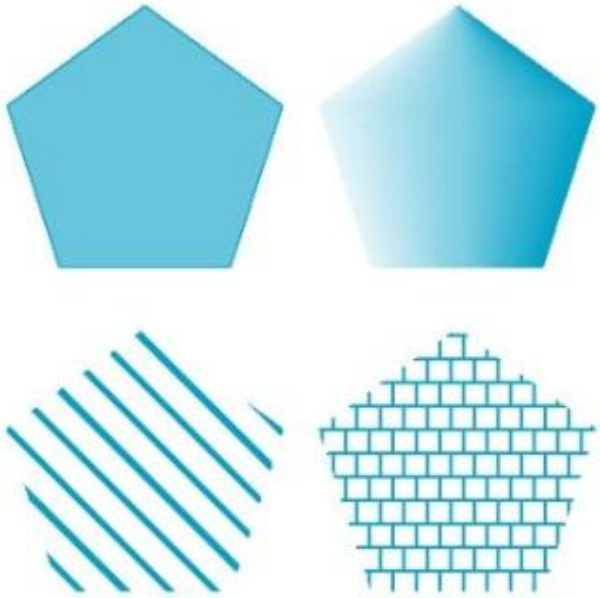
Polygon Basics

- Filled line-loop structure.
- Polygons play a special role in computer graphics because they can be displayed rapidly and they can be used to approximate arbitrary or curved surfaces.
- The performance of graphics systems is measured by the number of polygons per second that can be displayed.



Ways of displaying polygons

- Only its edges be displayed.
- Its interior be filled with a solid color, or a pattern.
- Either display or not display the edges.
- Outer edges of a polygon can be defined easily by an ordered list of vertices.
- But if the interior is not well defined, then the polygon may be rendered incorrectly.

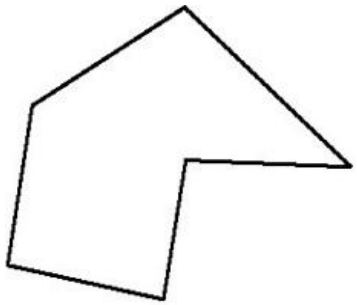


Properties of a polygon

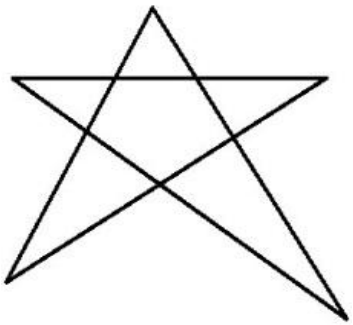
Three properties will ensure that a polygon will be displayed correctly. It must be **simple, convex, and flat.**

1. Simple:

- A 2-D polygon in which **no pair of edges cross each other.**
- They will have well-defined interiors.



Simple

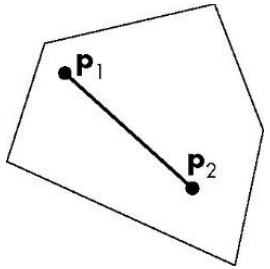


Non simple

Properties of a polygon

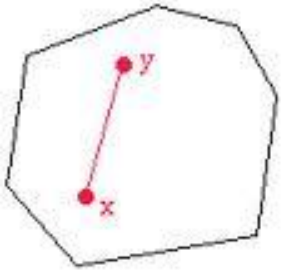
2. Convex

- A convex polygon is a multi-angled shape with straight sides in which all of the angles are **< 180^0**.
- A **pentagon** is an example of a convex polygon.
- A line drawn through a convex polygon will intersect the sides of the polygon **exactly twice.**
- All triangles and regular polygons such as squares, parallelograms, trapezoids, and rectangles are convex because each of their interior angles is less than 180^0.
- All the points between p1 and p2 will be inside the polygon **(on the surface of the polygon).**

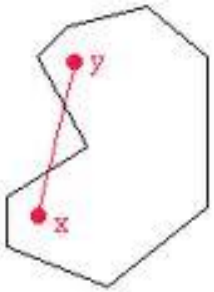


Properties of a polygon

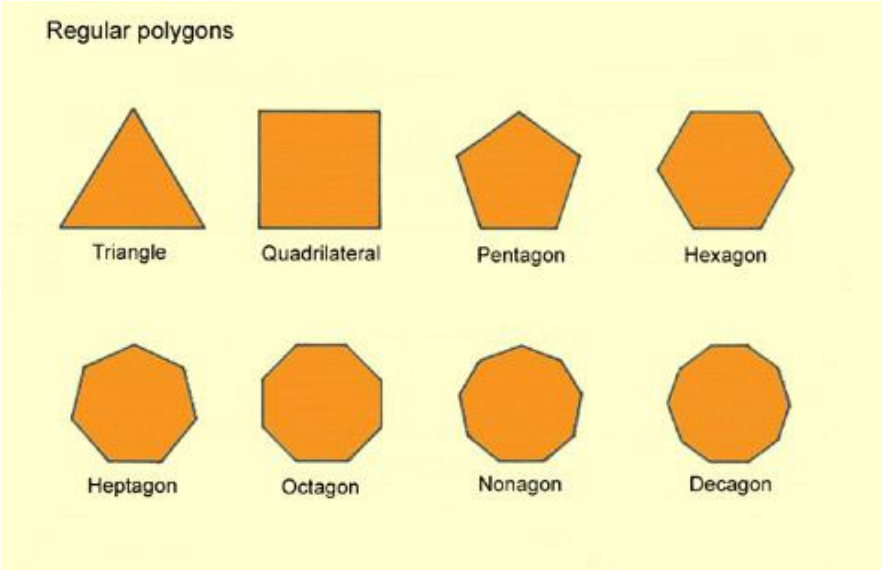
2. Convex



A convex polygon



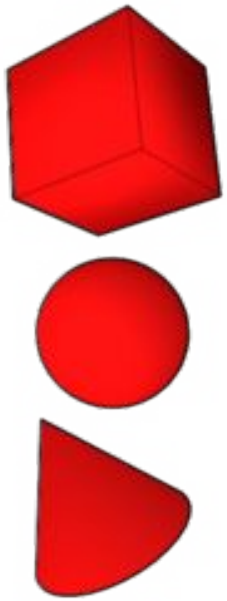
A non–convex polygon

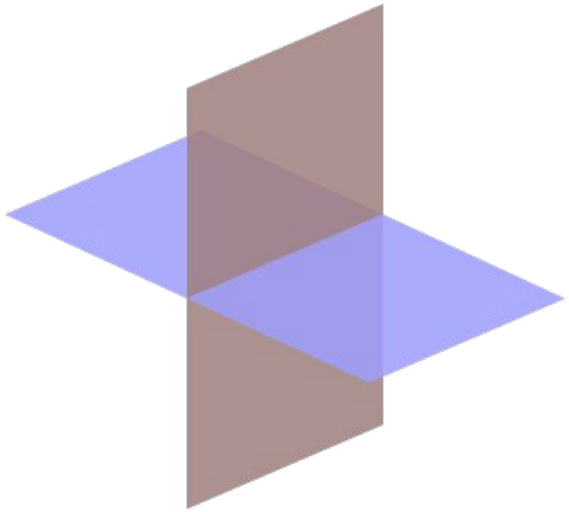


Properties of a polygon

3. Flat

- Flat defines that the vertices lie in the **same plane.** (A plane is a perfectly flat surface extending in all directions).
- In 3-D, **all vertices** that define polygon **need to lie in the same plane**.



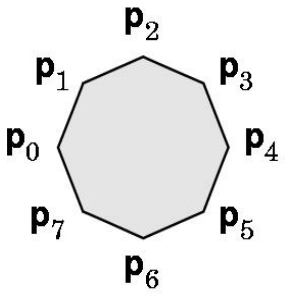


Polygon Types in OpenGL

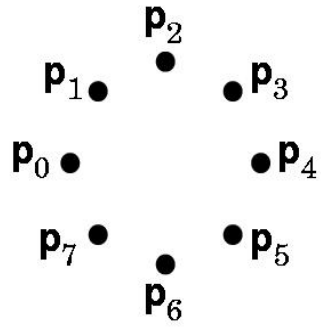
GL_POLYGON : Connects the successive vertices using line segments to form a closed path. The interior is filled according to the state of the relevant attributes.

GL_QUADS : Special case of polygon where successive group of 4 vertices are interpreted as quadrilaterals.

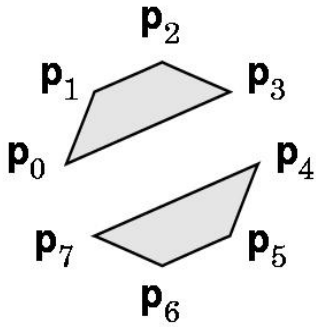
GL_TRIANGLES : Special case of polygon where successive group of three vertices would be interpreted as a triangle.



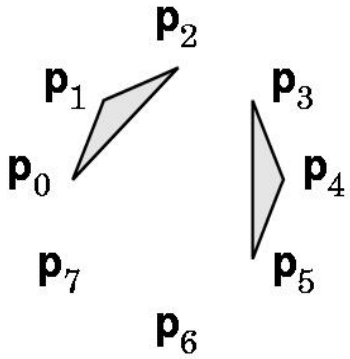
GL_POLYGON



GL_POINTS



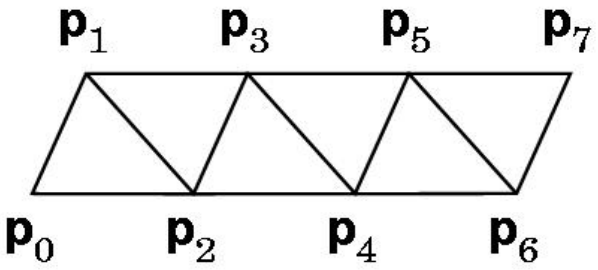
GL_QUADS



GL_TRIANGLES

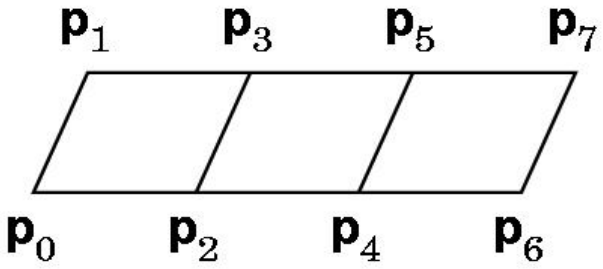
Polygon Types in OpenGL

GL_TRIANGLE_STRIP : Each additional vertex is combined with the previous two vertices to define a new triangle.



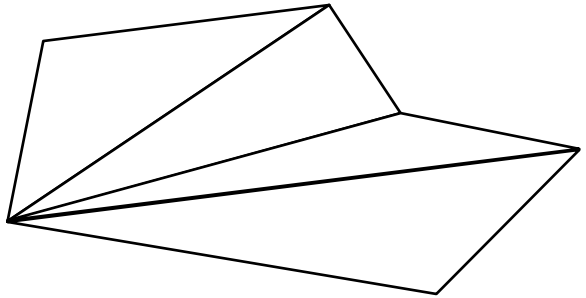
GL_TRIANGLE_STRIP

GL_QUAD_STRIP : Two new vertices are combined with the previous two vertices to design a new quadrilateral.



GL_QUAD_STRIP

GL_TRIANGLE_FAN : Based on one fixed point. The next two points determine the first triangle. The subsequent triangles are formed from one new point, the previous point and the first (fixed) point.

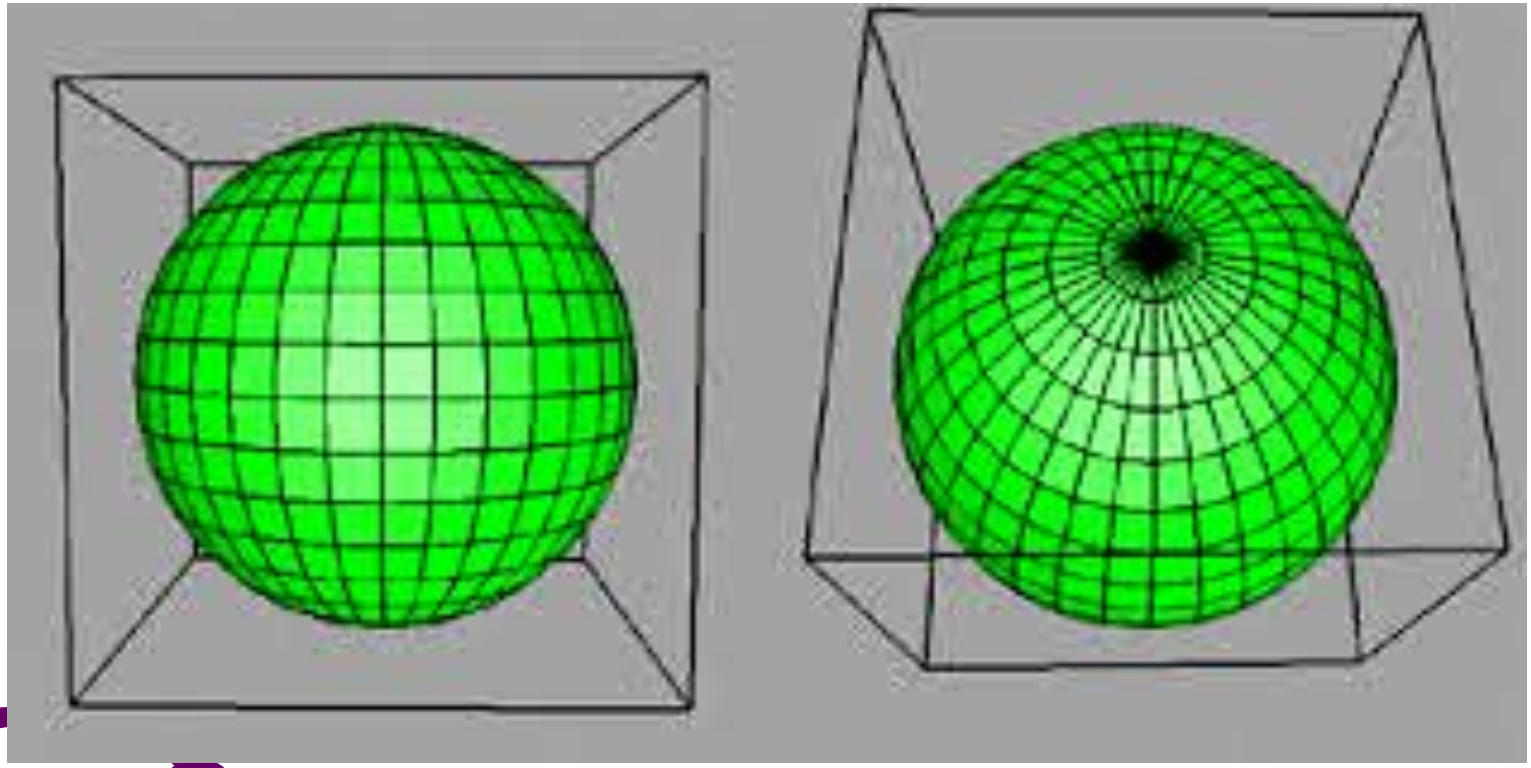


GL_TRIANGLE_FAN

Curved Primitives

- Curved primitives such as sphere, oval, parabola can be created using ***tessellation.***
- ***Tessellation*** is a process of creating curved primitives by using a mesh of polygon of n sides.
- ***Example*** : Approximating a Sphere

Approximating a Sphere



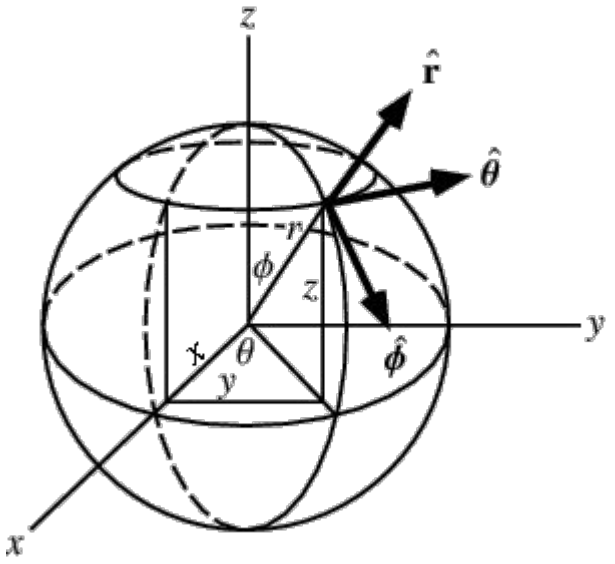
Approximating a Sphere

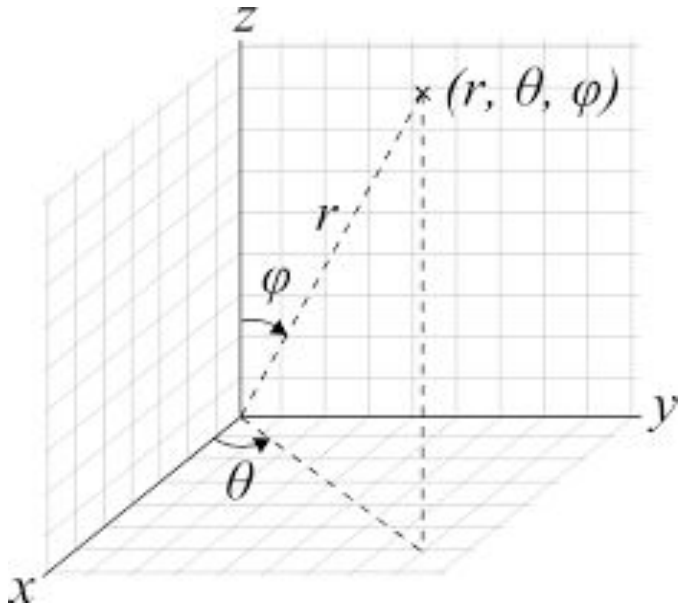
Approximating a Sphere

- Many curved surfaces can be approximated using fans and strips.
- E.g. To approximate to a sphere, **a set of polygons defined by lines of longitude and latitude** as shown can be used.
- Either quad strips or triangle strips can be used for the purpose.
- Consider a unit sphere (r=1). It can be described by the following three equations: $x(\theta, \emptyset) = \sin\theta \cos\emptyset$

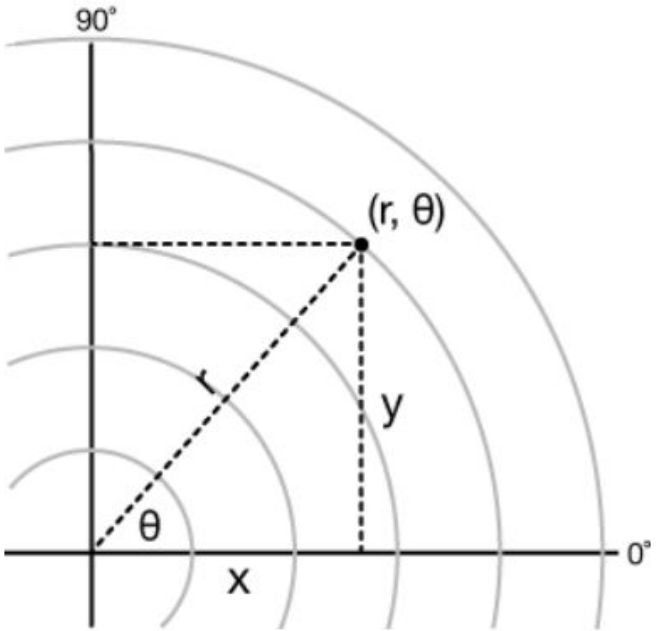
$y(\theta, \emptyset) = \cos\theta \cos\emptyset$

$z(\theta, \emptyset) = \sin\emptyset$



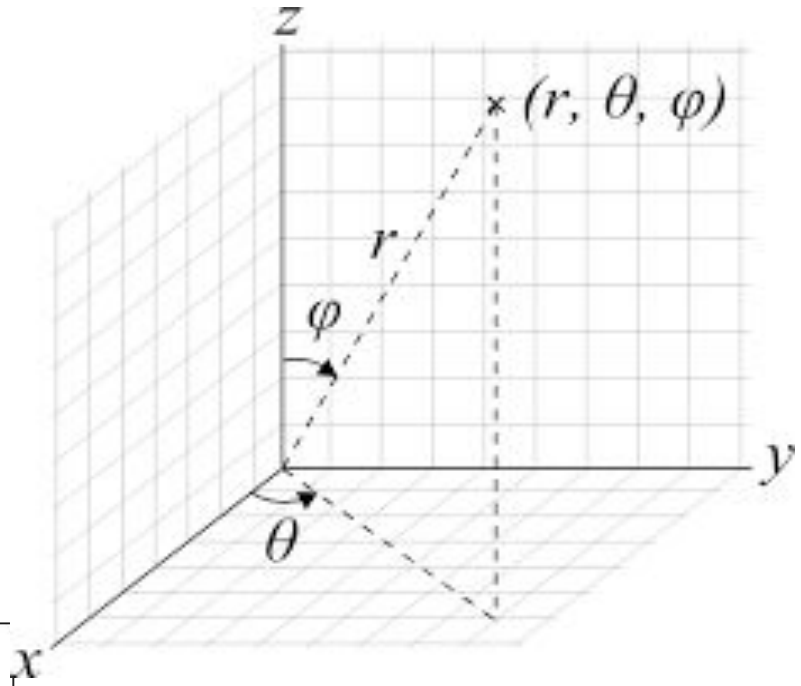


Approximating a Sphere



$$x = r\cos(\theta) \qquad y = r\sin(\theta)$$

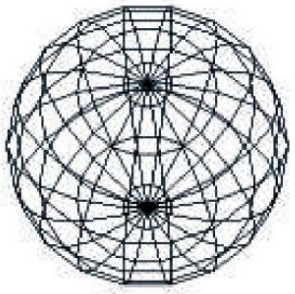
$$r^2 = x^2 + y^2 \qquad \theta = \tan^{-1}\left(\frac{y}{x}\right)$$

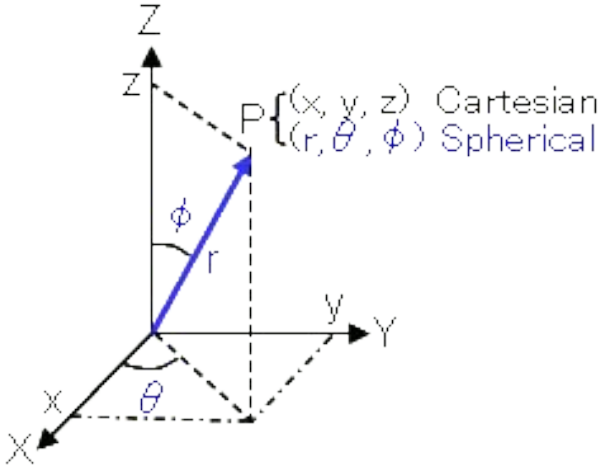


x(θ, Ø)= sinθ cosØ
y(θ, Ø)= cosθ cosØ
z(θ, Ø) = sinØ

Approximating a Sphere

$x(\theta, \emptyset) = \sin\theta \cos\emptyset$
$y(\theta, \emptyset) = \cos\theta \cos\emptyset$
$z(\theta, \emptyset) = \sin\emptyset$





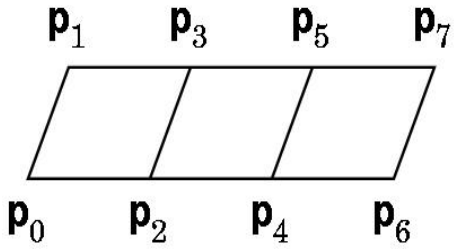
- If we **fix Ø** and draw curves, as we change θ, we get circles of **constant latitude.**
- If we **fix θ** and draw curves, as we change Ø, we get circles of **constant longitude.**
- By generating points at fixed increments of θ and Ø, we can define quadrilaterals.
- Ø must be ranged between -80 to +80.
- θ must be ranged between -180 and 180.

Approximating a Sphere

- Degrees must be converted to radians for the standard trigonometric functions.
- **Code :**

```
float c=3.142/180; // converting to radians
// to get longitudes
for(phi=-80.0; phi<=80.0; phi+=20.0)
{
    phir=c*phi;  // φ in radians – for the 1st point
    phir20=c*(phi+20);  // for the 2nd point
```

Approximating a Sphere




```
glBegin(GL_QUAD_STRIP);
// to get latitudes
for (theta=-180.0; theta<=180.0; theta+=20.0)
{
    thetar=c*theta;
    x=sin(thetar)*cos(phir);
    y=cos(thetar)*cos(phir);
    z=sin(phir);
    glVertex3d(x,y,z); // 1st point
```

Approximating a Sphere

```
        //phir20=c*(phi+20);
        x=sin(thetar)*cos(phir20);
        y=cos(thetar)*cos(phir20);
        z=sin(phir20);
        glVertex3d(x,y,z); // 2nd point
    }
    glEnd();
}
```

Approximating a Sphere

```
//1st pole – use triangle fans
glBegin(GL_TRIANGLE_FAN);
glVertex3d(0.0,0.0,1.0);   // top pole
c80=c*80.0;
z=sin(c80);
for(theta=-180.0;theta<=180.0;theta+=20.0)
{    thetar=c*theta;
     x=sin(thetar)*cos(c80);
     y=cos(thetar)*cos(c80);
     glVertex3d(x,y,z);
}
glEnd();
```

Approximating a Sphere

```
//2nd pole – use triangle fans
glBegin(GL_TRIANGLE_FAN);
glVertex3d(0.0,0.0,-1.0); // bottom pole
z=-sin(c80);
for(theta=-180.0;theta<=180.0;theta+=20.0)
{
    thetar=c*theta;
    x=sin(thetar)*cos(c80);
    y=cos(thetar)*cos(c80);
    glVertex3d(x,y,z);
}
glEnd();
```

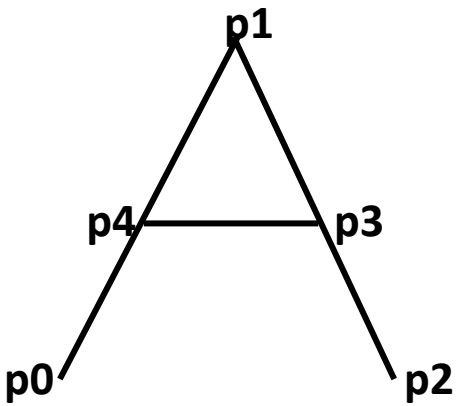
Text

Text

- In computer graphics, text may need to be displayed in a multitude of **fashions** by controlling **type styles, sizes, colors, and other parameters.**
- Fonts - Times, Computer Modern or Helvetica.
- Two types
  - **Stroke text**
    - **void glutStrokeCharacter(void *font, int character)**
  - **Raster text**
    - **void glutBitmapCharacter(void *font, int character)**

Stroke Text

- It is constructed **similar to geometric objects.**
- Vertex is used to define the line segment or curves that outline each character.
- The characters defined by closed boundaries can be filled.
- Stroke fonts behave like any 3D object, i.e. the characters can be rotated, scaled, and translated.



Stroke Text

Advantages

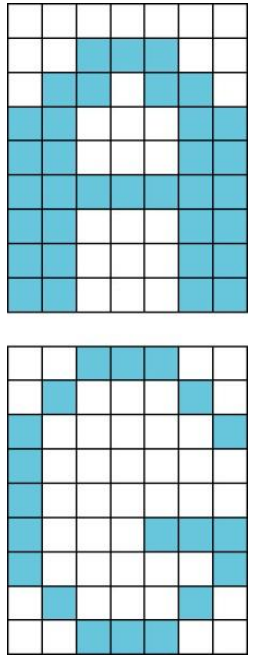
- It can be translated, rotated, scaled etc.
- It can be viewed using different views.

Disadvantages

- Consumes lot of memory.
- Consumes lot of processing time.

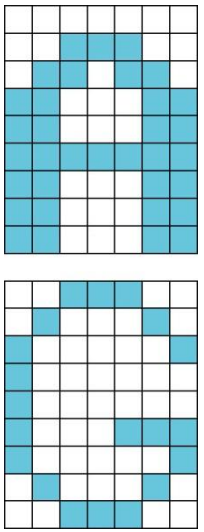
Raster text

- It is simple and fast.
- Characters are defined as rectangles of bits called **bit blocks**.
- **Each block defines a single character by the pattern of 0 and 1 bits in the block.**
- A raster character can be placed in the frame buffer rapidly by a **bit-block-transfer operation**, which **moves the block of bits using a single instruction.**



Raster text

- OpenGL allows the application program to use instructions that allow direct manipulation of the contents of the frame buffer.
- **Size of raster characters** can be **increased only by replicating or duplicating pixels**, a **process that gives larger characters a blocky appearance.**



Advantages

- Can be placed in the frame buffer rapidly by a bit block transfer.
- Simple and fast.

Disadvantages

- Rotation, scaling etc cannot be applied.
- Raster characters often are stored in read-only memory (ROM) in the hardware and hence, a particular font might be of limited portability.

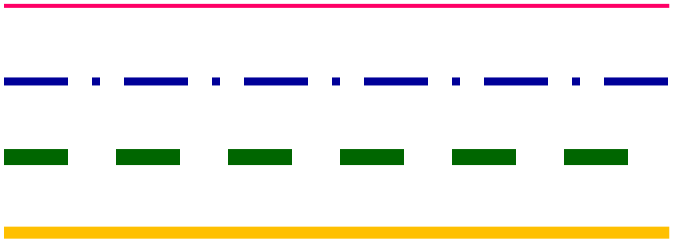
Attributes (July 2011)

- Property of a geometric primitive that refers to its ***color, thickness, pattern, edged, non-edged*** etc.
- Each geometric type has a set of attributes such as
  - A **point** has **color** and **thickness** attributes.
  - A **line** has **color, thickness** and **pattern** attributes.
  - A **polygon** has **color, thickness, pattern, display only edges, display without edges attributes.**
- **Type of the primitive differs from its attribute**.

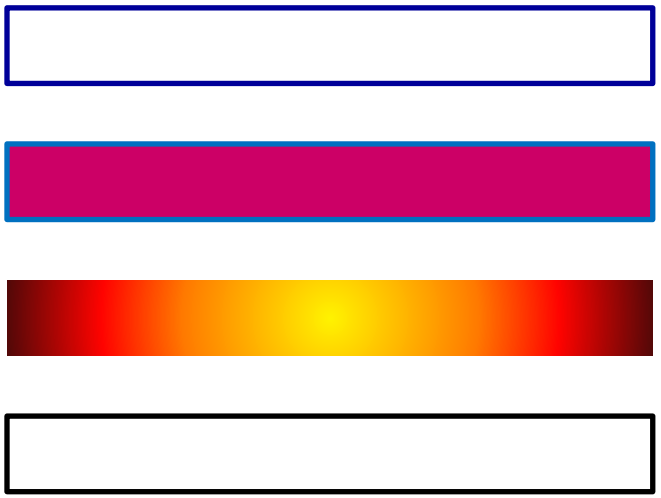
  E.g. ***A red solid line and a green dashed line are the same geometric type, but each is displayed differently.***
- Attribute defines the nature of the geometric primitive and appearance of objects. ***glColor3f(); glColorClear();***

Attributes for lines and polygons

Lines



Polygons



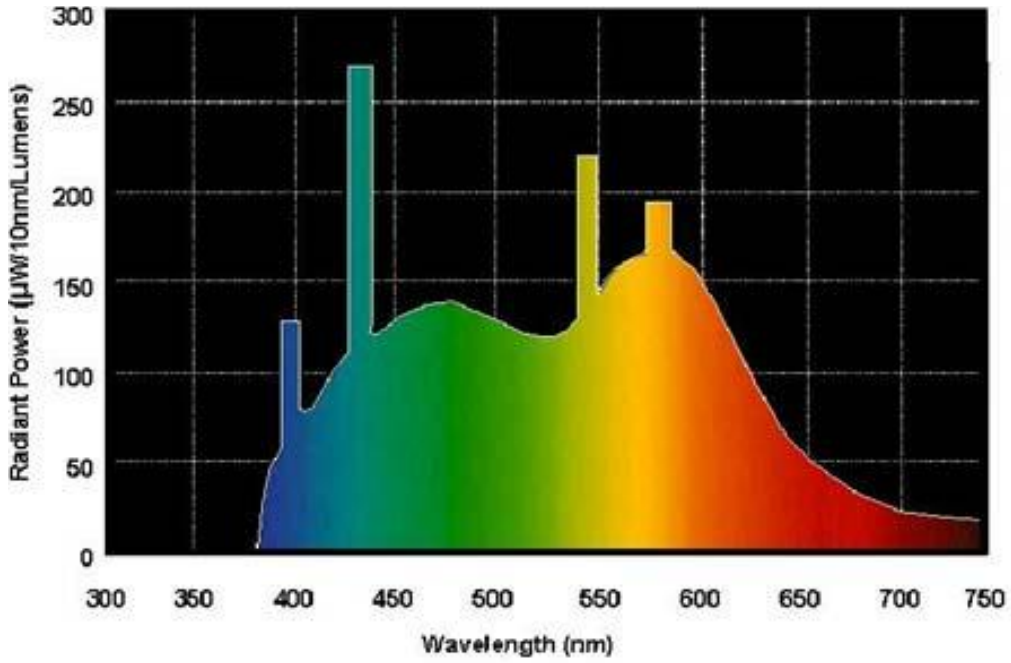
Immediate-mode graphics

- In this, ***primitives are not stored in the system***, but rather are ***passed through the system for possible display as soon as they are defined***.
- The present values of attributes are part of the state of the graphics system.
- When a primitive is defined, the present attributes for that type are used, and it is displayed immediately.
- There is no memory of the primitive in the system. Only the primitive's image appears on the display; once erased from the display, it is lost.

Color

Color

- It is the most important attribute of any geometric primitive.
- A visible color can be characterized by function C(X) that occupies wavelengths from about 350 nm to 780 nm.



Color

- The human visual system has 3 types of cones responsible for color vision.
- Hence our brains do not receive the entire distribution for the given color but 3 values – **tristimulus values** – that are responses of the **3 types of cones to the color.**
- This reduction of a color to 3 values leads to the basic tent of three-color theory.
- If two colors produce same tristimulus values, then they are visually indistinguishable.

Tristimulus Values

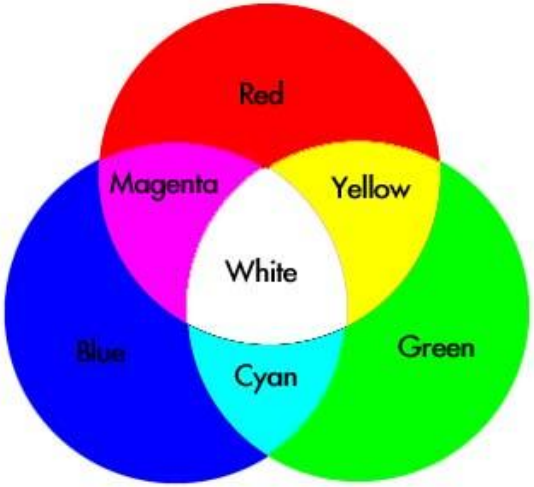
- Any color which can be produced by three primary colors - blue, green, and red and can be written as:

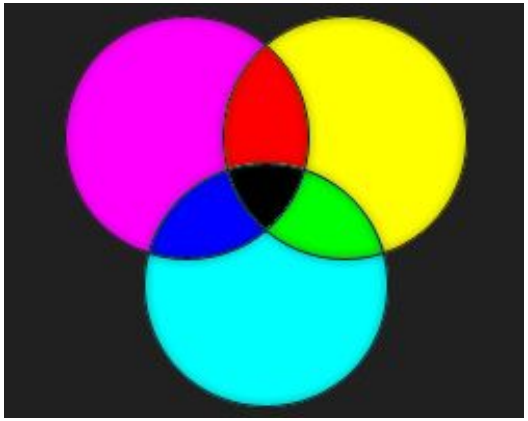
$$C = rR + gG + bB$$

- Where **R,G,B** can be considered to be "unit values" for blue, green, and red.
- r,g,b are the magnitudes or relative intensities of those primaries and are called "tristimulus values".

Color modes

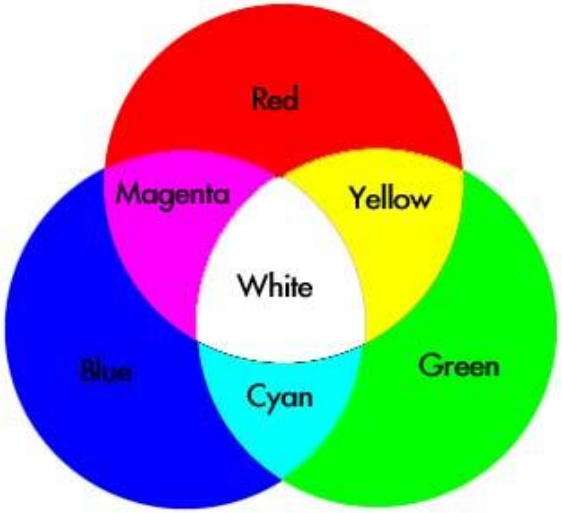
- **Additive color**
- **Subtractive color**

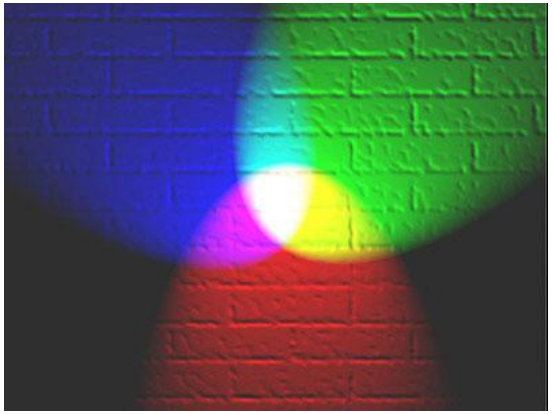




Additive Color

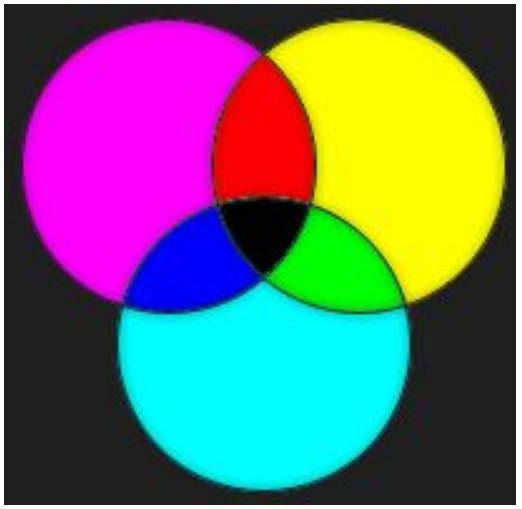
- **Primary** colors used **(RGB)**
  - **Red**
  - **Green**
  - **Blue**.
- CRT is an example of **additive color** where primary colors are added together to give the perceived color.

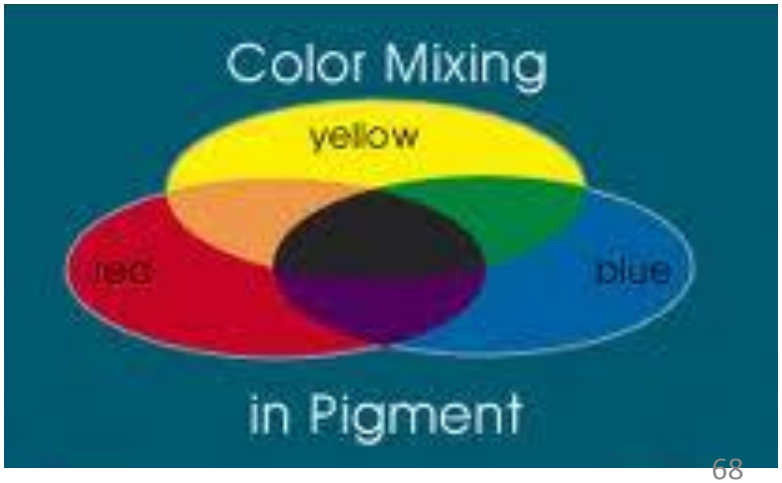




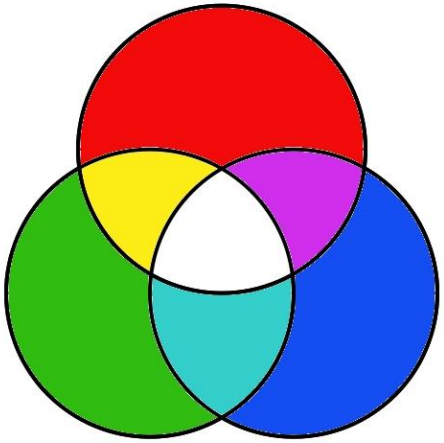
Subtractive Color

- It suits for processes such as **commercial printing and painting** as white surface is used as background.
- The primaries are usually the complementary colors (**CMY**)
  - **Cyan**
  - **Magenta**
  - **Yellow**

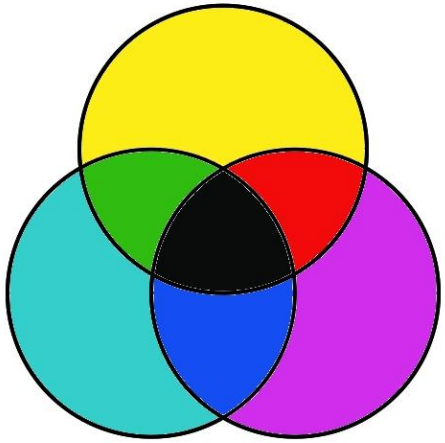


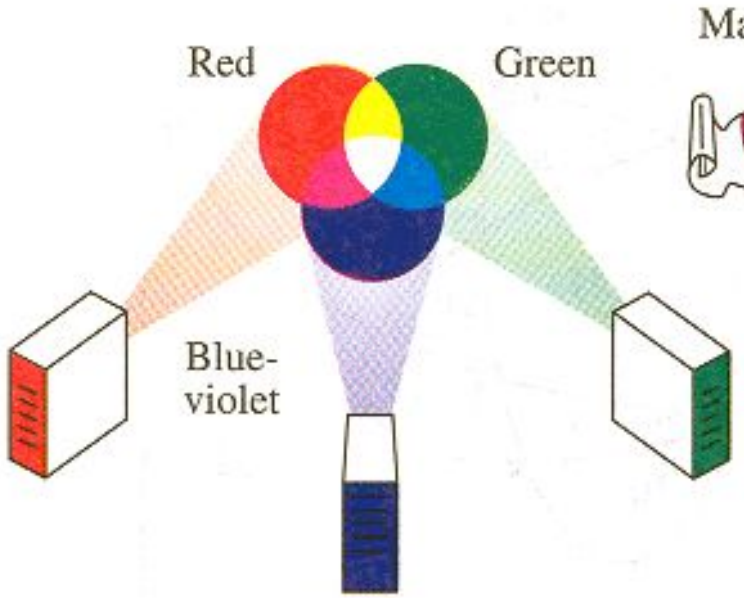


ADDITIVE

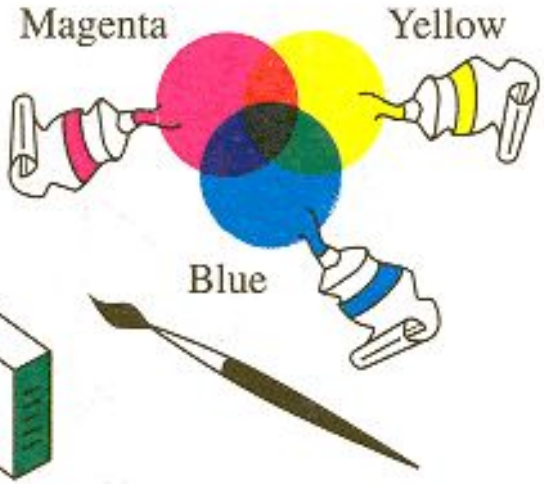


SUBTRACTIVE





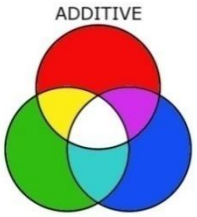
a. Additive color

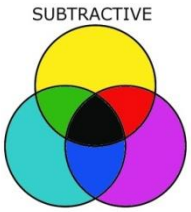


b. Subtractive color

Differences between Additive color and Subtractive color

Additive Color	Subtractive Color
Used in CRTs.	Used for commercial printing and painting.
Primary colors are Red, Green, Blue (RGB).	Primary colors are Cyan, Magenta, Yellow (CMY).
Primaries add light to an initially **black** display to give desired color.	Primaries subtract color from an initially **white** display to give desired color.

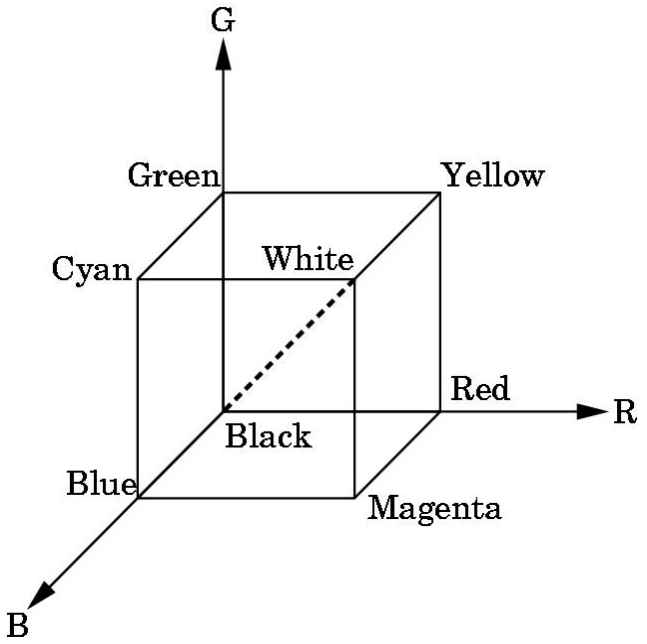


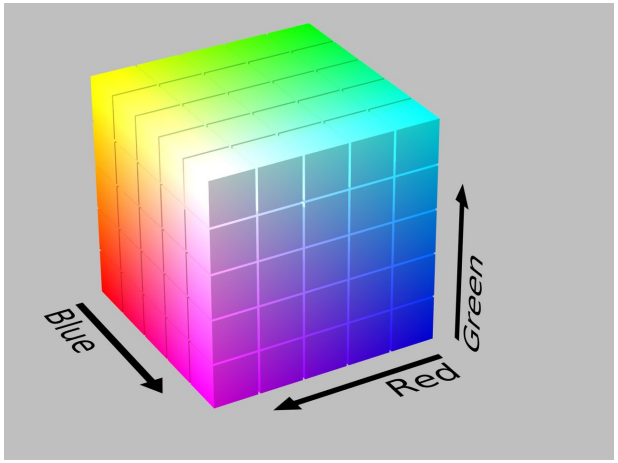


- **We can view color as a point in a color solid**
- **The vertices of a cube correspond to:**

  black(no primaries on)
  red, **green**, **blue**(one primary fully on)
  cyan(green and blue fully on)
  magenta(red and blue fully on)
  yellow(red and green fully on)
  white(all primaries fully on)

- **All colors along principal diagonal line have equal tristimulus values and appear as shades of gray.**





Color Models

There are 2 color models

1. **RGB color model**
2. **Index color model**

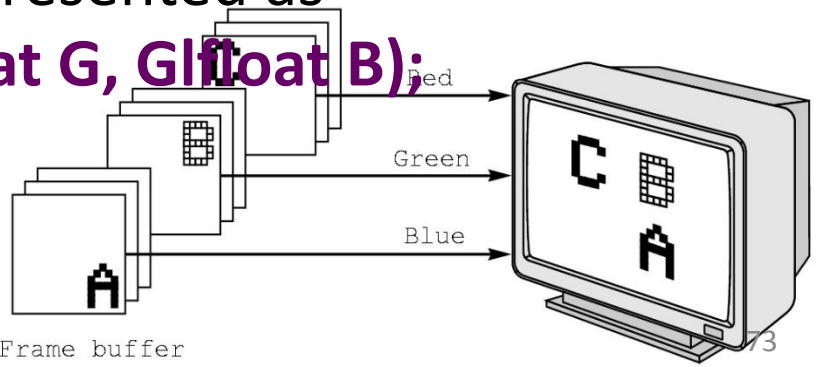
RGB Color

- In **additive color** RGB system, there are **separate buffers** for Red, Green and Blue images.
- Each pixel has separate R, G, and B components that corresponds to location in memory.
- **Ex : 24 bits for each pixel will be divided into**

  8 bits + 8 bits + 8 bits = 24 bits

  R G B
- Using 24 bit format, a total of 2^{24} different colors are possible.
- In OpenGL, color wrt RGB is represented as

  void glColor3f(GLfloat R, GLfloat G, Glfloat B);



RGB Color

- In OpenGL, the color attribute can be specified as a number between 0.0 and 1.0
- Ex : to draw in red, the following function call is used:

  glColor3f(1.0, 0.0, 0.0);

  Default value of each one is 0.
- The execution of this function will set the present drawing color to red. Because the color is part of the state, draw in **red is continued until the color is changed.**
- The "**3f**" is used in a manner similar to the **glVertex** function: It conveys that, a three-color (RGB) model is used, and that the values of the components are given as floats in C.

RGBA Color

- Four color model namely RGBA system.
- A(alpha) – used for creating ***fog effects***(combining the images).

  0.0 – transparent (passes all kinds of light)

  1.0 – opaque(does not pass any light)

glClearColor(1.0,1.0,1.0,1.0)

would set the background color of the window to white and opaque.

Indexed Color

- Commonly used color model.
- It is used when **frame buffer has limited depth** *(No. of bits per pixel).*
- ***Example :***

  If the frame buffer has 1280 x 1024 array of pixels but with each pixel consisting of only 8 bits, then color index model is used.

Input	Red	Green	Blue
0	0	0	0
1	$2^m - 1$	0	0
.	0	$2^m - 1$	0
.	.	.	.
.	.	.	.
$2^k - 1$	.	.	.
	m bits	m bits	m bits

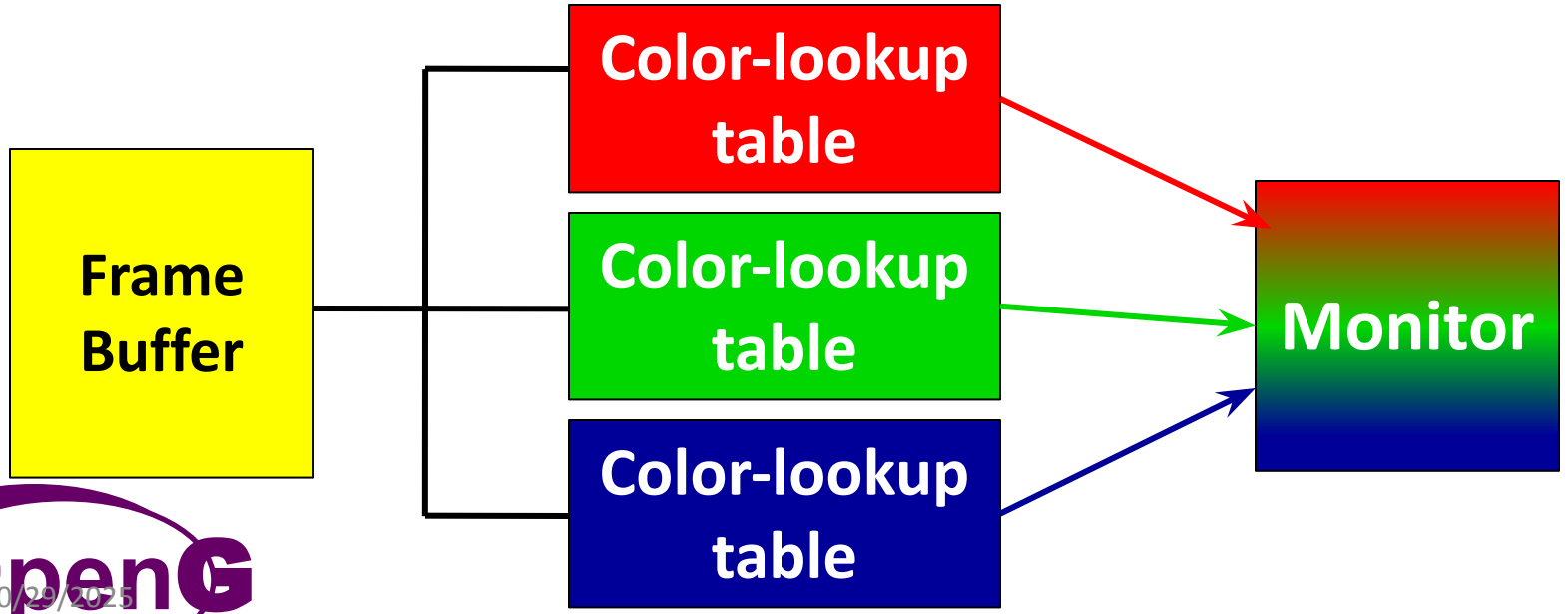
Indexed Color

Input	Red	Green	Blue
0	0	0	0
1	$2^m - 1$	0	0
.	0	$2^m - 1$	0
.	.	.	.
.	.	.	.
$2^k - 1$	.	.	.
	m bits	m bits	m bits

- In the model, suppose there are 'k' bits per pixel, then each pixel value is an integer between 0 to $2^k - 1$.
- We can display colors with a precision of m bits.
- We can choose from 2^m reds, 2^m greens and 2^m blues.
- Hence we can produce any of 2^{3m} colors on the display but the frame buffer can specify only 2^k of them.
- glIndexi(element);
- glutSetColor(int color, Glfloat red, Glfloat green, Glfloat blue);

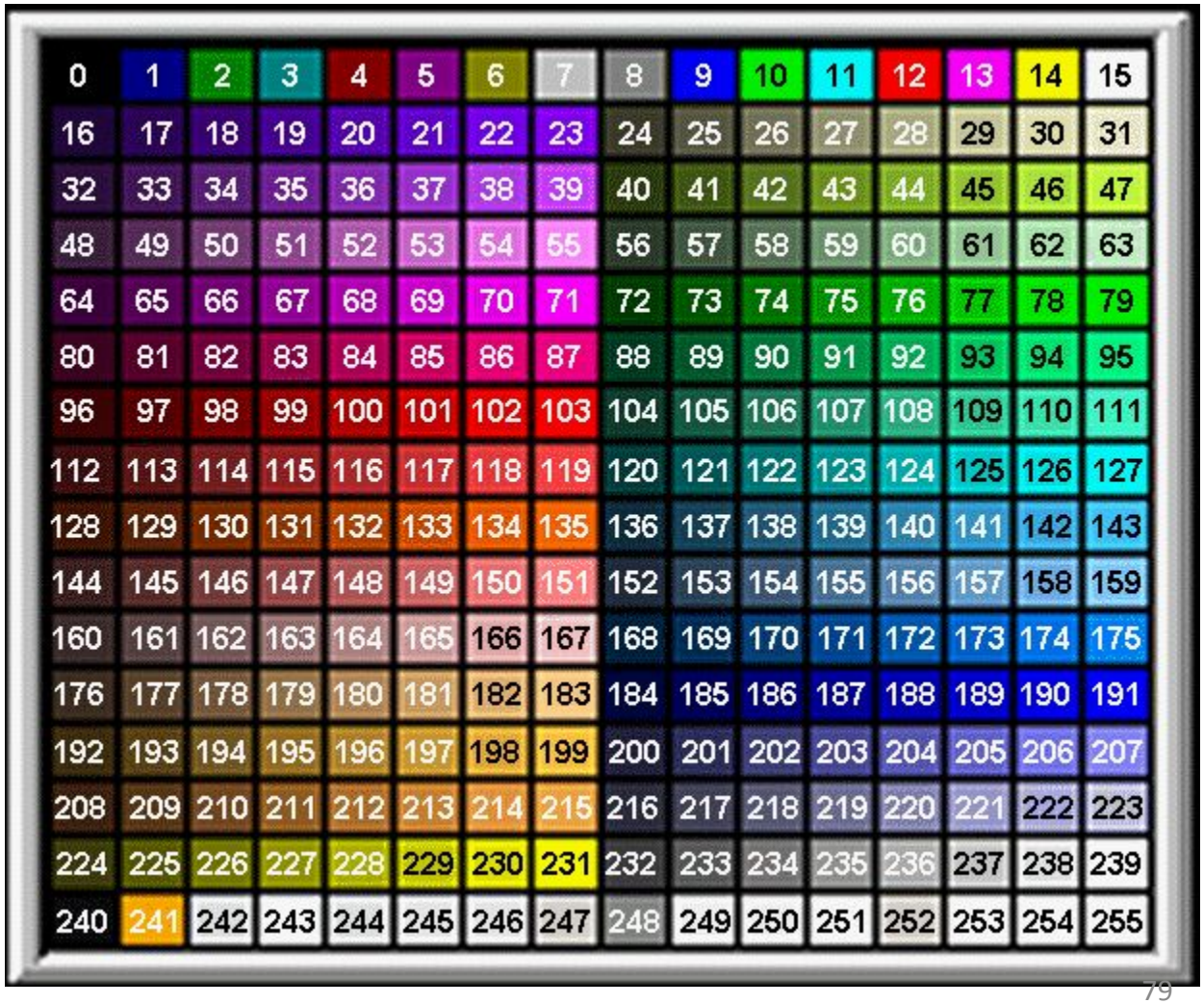
Indexed Color

Input		Red	Green	Blue
0		0	0	0
1		$2^m - 1$	0	0
.		0	$2^m - 1$	0
.		.	.	.
.		.	.	.
$2^k - 1$		.	.	.
		m bits	m bits	m bits

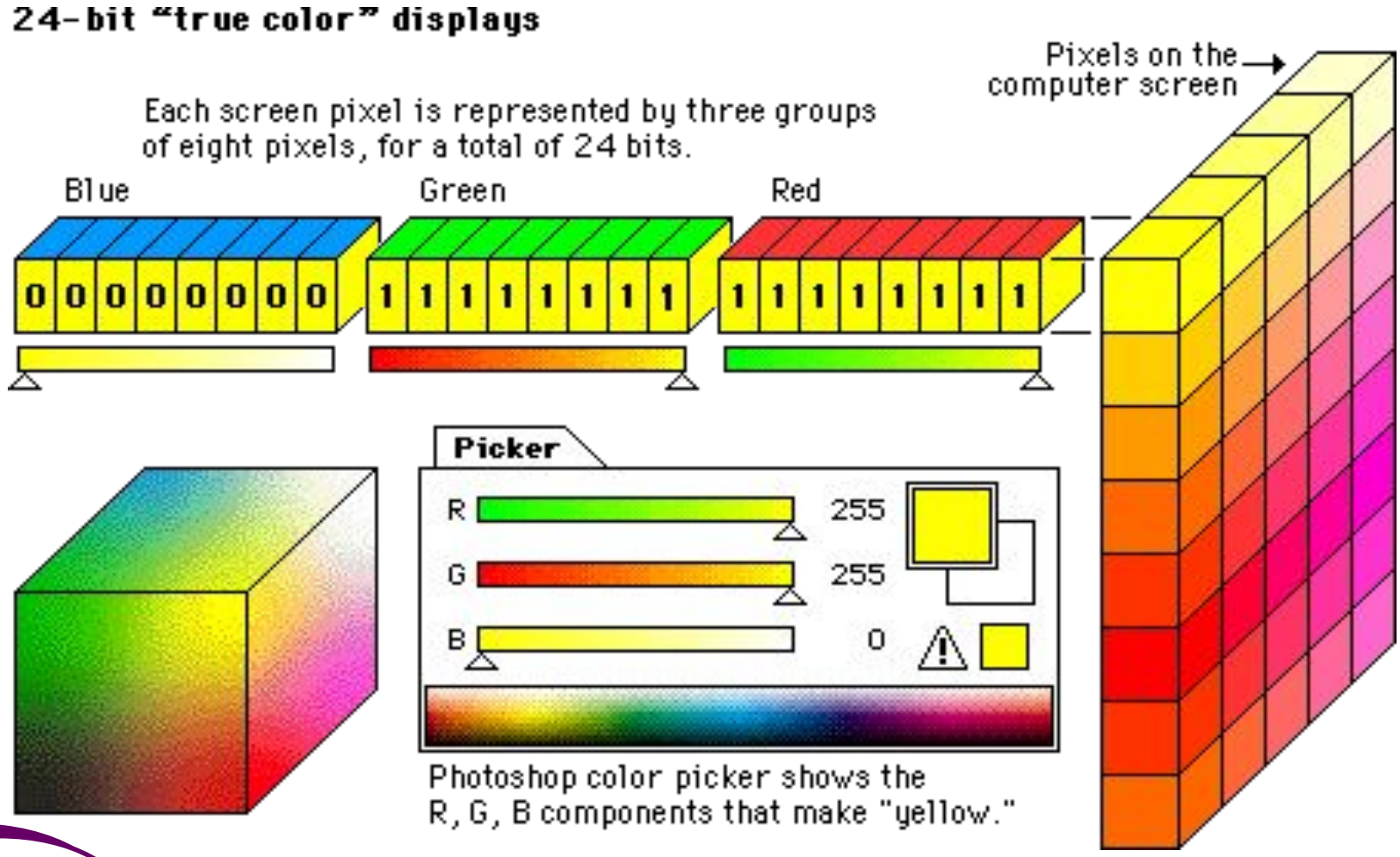


Indexed Color palette

If k=8 bits per pixel, we get $2^8 - 1 = 256$ colors palette.



Indexed Color palette



Indexed Color

Advantages

- We can operate with such systems where the **number of bits per pixel is less.**
- The programmer can change the lookup table according to his requirements and hence has the flexibility to enjoy the usage of all possible colors.

Disadvantages

- All possible colors would not be available for usage simultaneously.

Setting of Color attributes

glClearColor(1.0,1.0,1.0,1.0);
Background color is set to white.

glColor3f(0.0, 1.0, 0.0);
Rendering color for the points is set to Red.

glPointSize(2.0);
The size of rendered points is set to 2 pixels wide.

Control Functions

Control Functions

Control functions enable the programmer to communicate with the window system, initialize the program etc.

1. glutInit(int *argcp, char **argv)
2. glutCreateWindow(char *title)
3. glutInitDisplayMode()
4. glutInitWindowSize()
5. GlutInitWindowposition()
6. glutDisplayFunc()
7. glutMainLoop()

Control Functions

1. **glutInit(int *argcp, char **argv)**
   - It initiates an interaction between the windowing system and OpenGL. It is used before opening a window in the program.
   - The two arguments allow the user to pass command-line arguments, as in the standard C main function, and are usually the same as in main.

3. **glutCreateWindow(char *title)**
   - It opens an OpenGL window.
   - The title string provides title at the top to the window displayed.

Control Functions

3. **glutInitDisplayMode()**
   Used to initialize the display window created, by specifying color model, buffering used etc.

4. **glutInitWindowSize()**
   Used to specify the size of the window to be created.

5. **glutInitWindowPosition()**
   Used to specify the position of the created window on the monitor with respect to the top left corner.

Control Functions

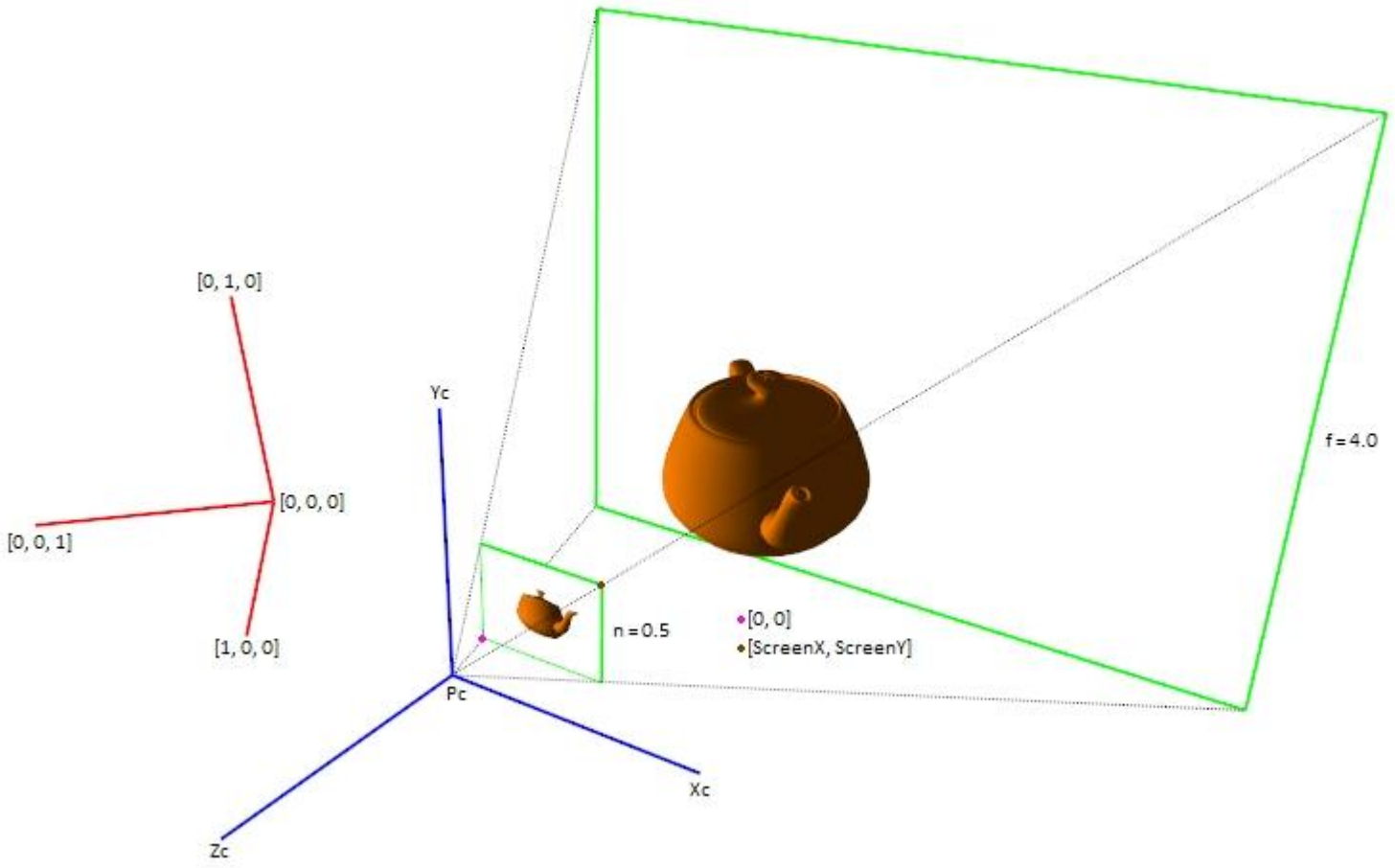
6. glutDisplayFunc()

- Used to specify the display call back.
- This function executes when the window is created for the first time.
- It is also called when the window is moved from one location on the screen to another.

7. glutMainLoop()

- Causes the program to begin an event processing loop.
- If there are no events to process, then the program would enter the wait state with the output on the screen.
- It is similar to getch() in C program.

Viewing

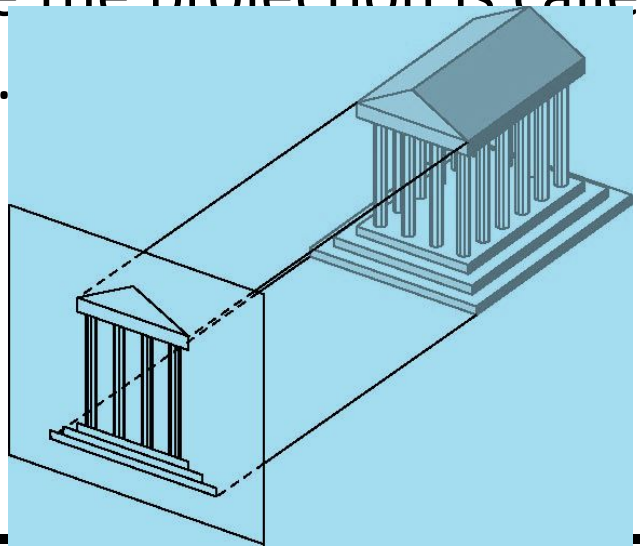


Viewing

- Viewing is a type of graphics functions which enables the programmer to specify the view facilities such as top view, front view, left side view, right side view, zoom in and zoom out.
- Determines the appearance of the object on the output device.
- If the programmer does not specify a view explicitly, **default one(orthogonal view) is considered.**
- Viewing can be
  - Orthographic view
  - 2D view
  - Matrix mode

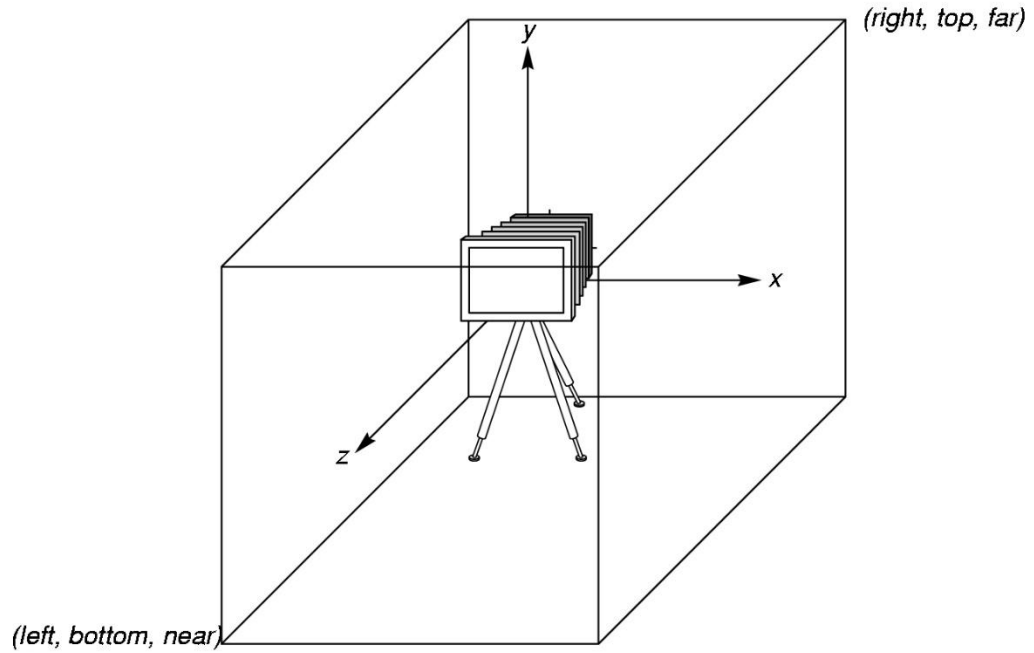
Orthographic View

- Simplest and OpenGL's **default view** is orthographic projection
- Can be obtained in the synthetic camera model by placing the camera at an **infinitely far distance** from the object.
- The projections(lines) are perpendicular(orthogonal) to the projection plane and hence the projection is called as orthogonal projection/viewing.



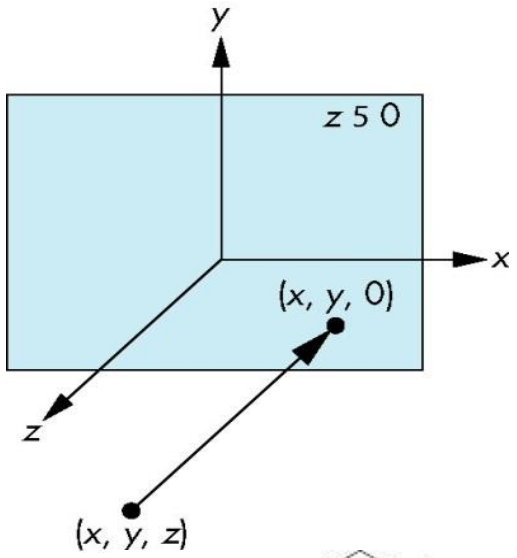
Orthographic View

- OpenGL places a camera at the origin in object space pointing in the negative z direction
- The default viewing volume is a box centered at the origin with a side of length 2.



Orthographic View

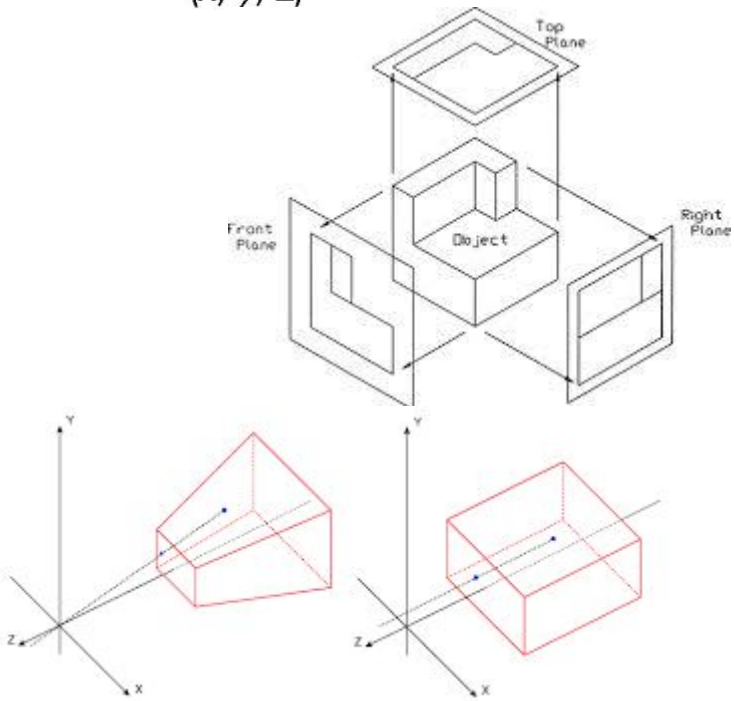
- **Orthographic projection takes a point (x,y,z) and projects it into the point (x,y,0)**
- **In 2-D viewing, with all vertices in plane z=0, a point and its projection both are same.**



In OpenGL, an orthographic projection with a right parallel piped viewing volume is specified via the following

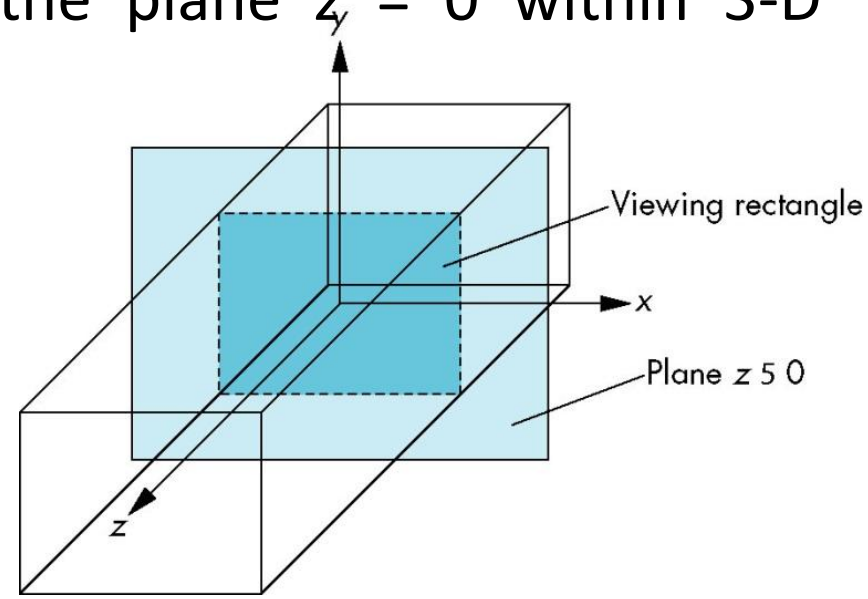
void glOrtho(Gldouble left, Gldouble right, Gldouble bottom, Gldouble top, Gldouble near, GLdouble far)

Sees only those objects in the volume specified by the viewing volume.



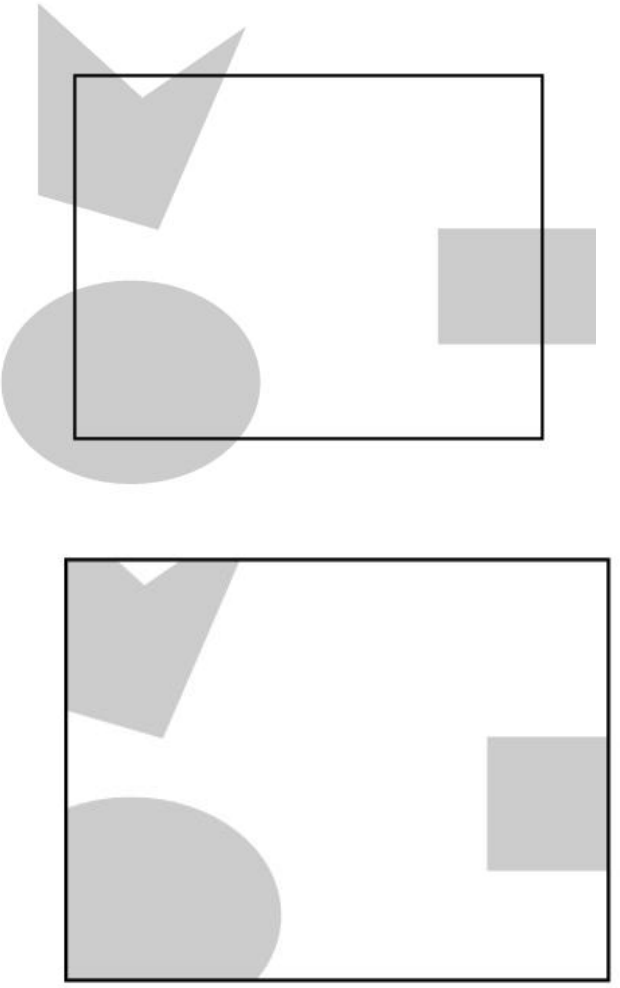
Orthographic View

- If viewing volume is not specified, OpenGL uses its default, a 2 x 2 x 2 cube, with the origin in the center. In terms of 2-D plane, the bottom-left corner is at(-1.0, -1.0), and the upper-right corner is at (1.0, 1.0).
- 2-D graphics view is a special case of 3-D graphics. Hence the viewing rectangle is in the plane $z = 0$ within 3-D viewing volume:



2-D viewing

- ***2-D viewing:*** is based on taking a rectangular area of 2-D world and transferring its contents to the display.
- The area of the world of which the image is to be taken is known as the *viewing rectangle* or *clipping rectangle*.
- Objects inside the rectangle are in the image; objects outside are clipped out and are not displayed.



2-D viewing

- Objects that straddle the edges of the rectangle are partially visible in the image.
- If using a 3-D volume seems strange in a two-dimensional application, the function

void gluOrtho2D(GLdoub1e left, GLdouble right, GLdouble bottom, GLdouble top)

in the utility library may make the program more consistent with its 2-D structure. This function is equivalent to **glOrtho** with near and far set to -1.0 and 1.0, respectively.

Matrix modes

- Pipeline graphics systems have an architecture that depends on multiplying together, or concatenating, a number of transformation matrices to achieve the desired image of a primitive.
- The two most important matrices are
  - Model-view
  - Projection matrices
- At any time, the state includes values for both of these matrices, which start off set to identity matrices.
- The usual sequence is to modify the initial identity matrix, by applying a sequence of transformations.

Matrix modes

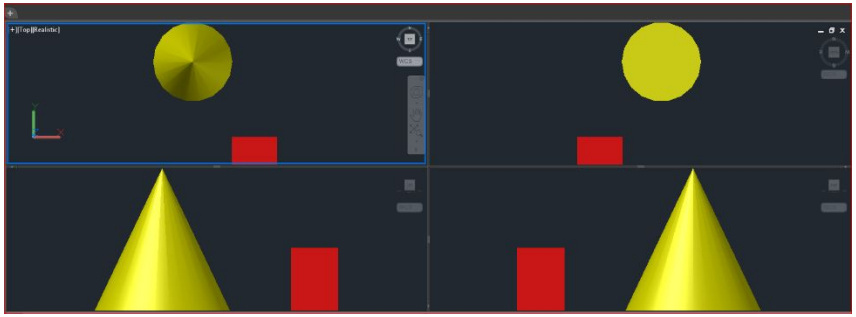
- Setting the matrix mode selects the matrix to which the operations apply. It is a variable that is set to one type of matrix and is also part of the state.
- The default matrix mode is to have operations apply to the model-view matrix.
- So to alter the projection matrix, modes must be switched. The following sequence is common for setting a 2-D viewing rectangle:
- **glMatrixMode(GL_PROJECTION);**
- **glLoadIdentity( );**
- **gluOrtho2D(0.0, 500.0, 0.0, 500.0);**
- **glMatrixMode(GL_MODELVIEW);**

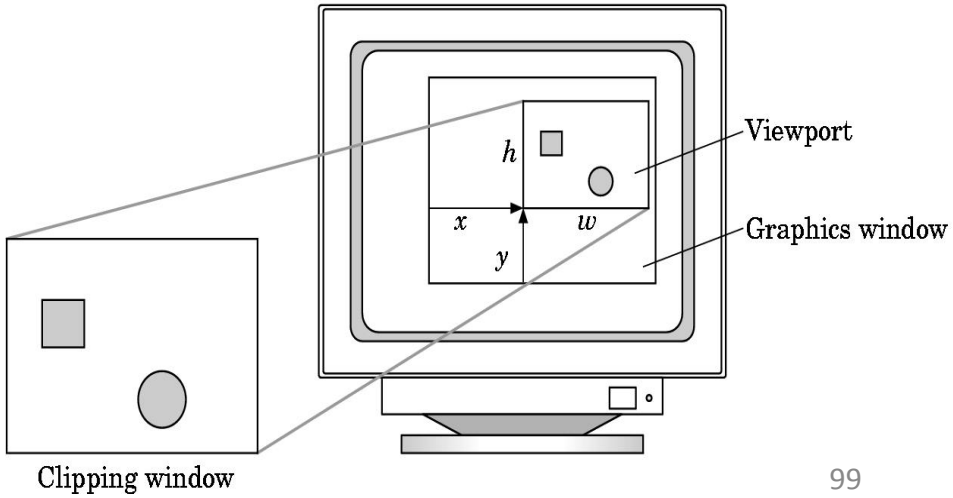
Matrix modes

- **glMatrixMode(GL_PROJECTION);**
- **glLoadIdentity( );**
- **gluOrtho2D(0.0, 500.0, 0.0, 500.0);**
- **glMatrixMode(GL_MODELVIEW);**
- This sequence defines a 500 x 500 viewing rectangle with the lower-left corner of the rectangle at the origin of the 2-D system.
- It then switches the matrix mode back to model-view mode.
- In complex programs, it is always better to return to a given matrix mode, to avoid possible confusion over the current mode. E.g. In this case mode returned to model-view mode.

Aspect Ratio and Viewports

- ***Aspect ratio*** : ratio of the rectangle's width to its height.
- ***Viewport*** : Rectangular area of the display window.
- By default, it is the entire window, but can be set to any smaller size in pixels through function.





Aspect Ratio and Viewports

void glViewport(GLint x, GLint y, GLsizei w, GLsizei h)

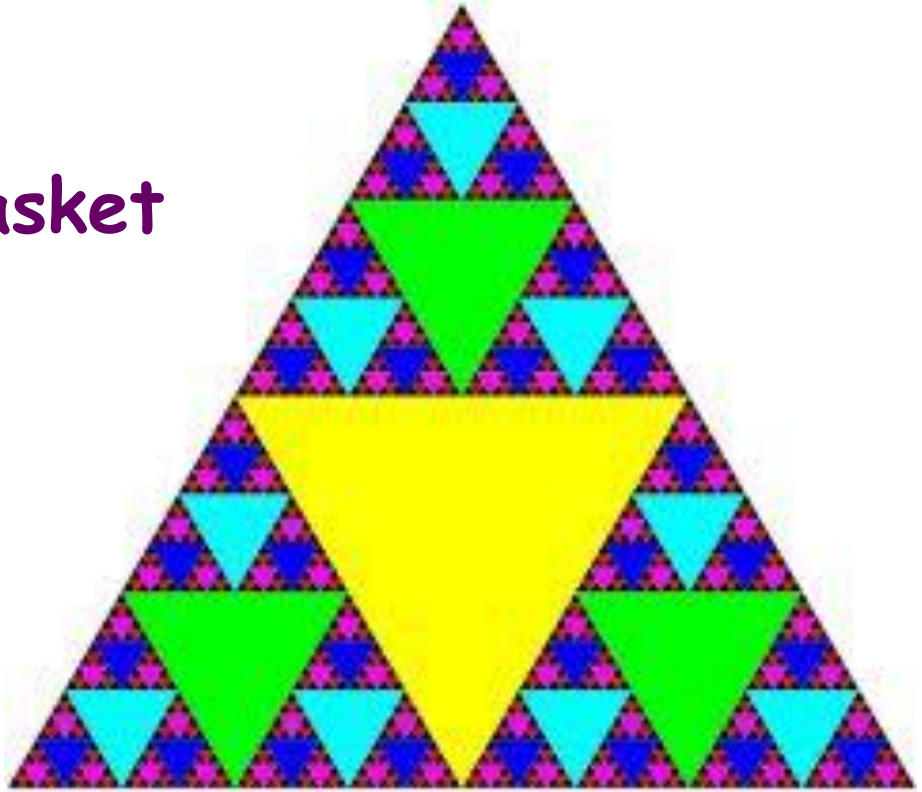
(x,y)

- Specify the lower left corner of the viewport rectangle, in pixels.
- The initial value is (0,0).

w and h (width and height)

- Specify the width and height of the viewport.
- When a GL context is first attached to a window, width and height are set to the dimensions of that window.

Sierpinski Gasket Program

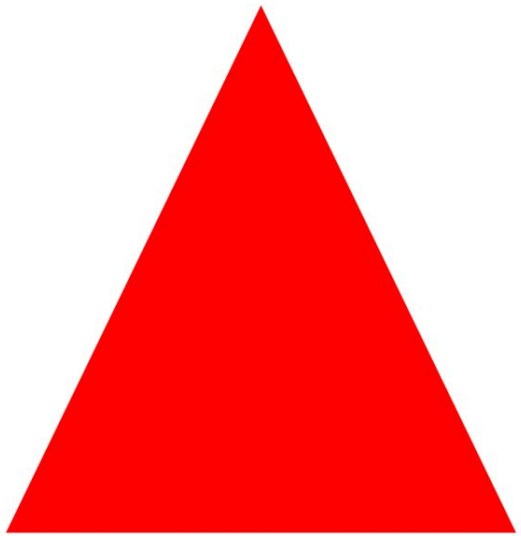


PROGRAM 3

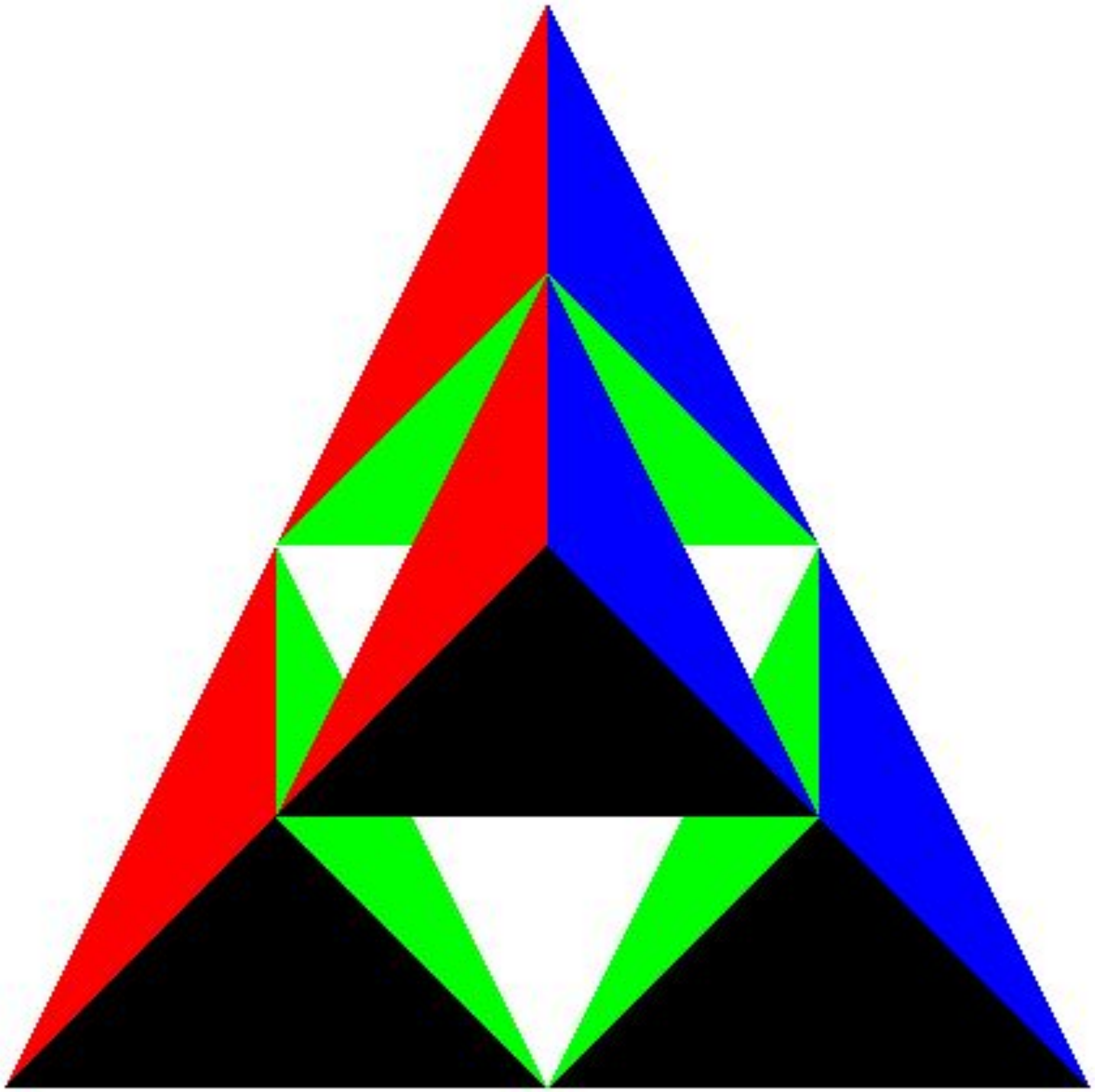
Write a program to recursively subdivides a tetrahedron to form 3D Sierpinski gasket. The number of recursive steps is to be specified at execution time.

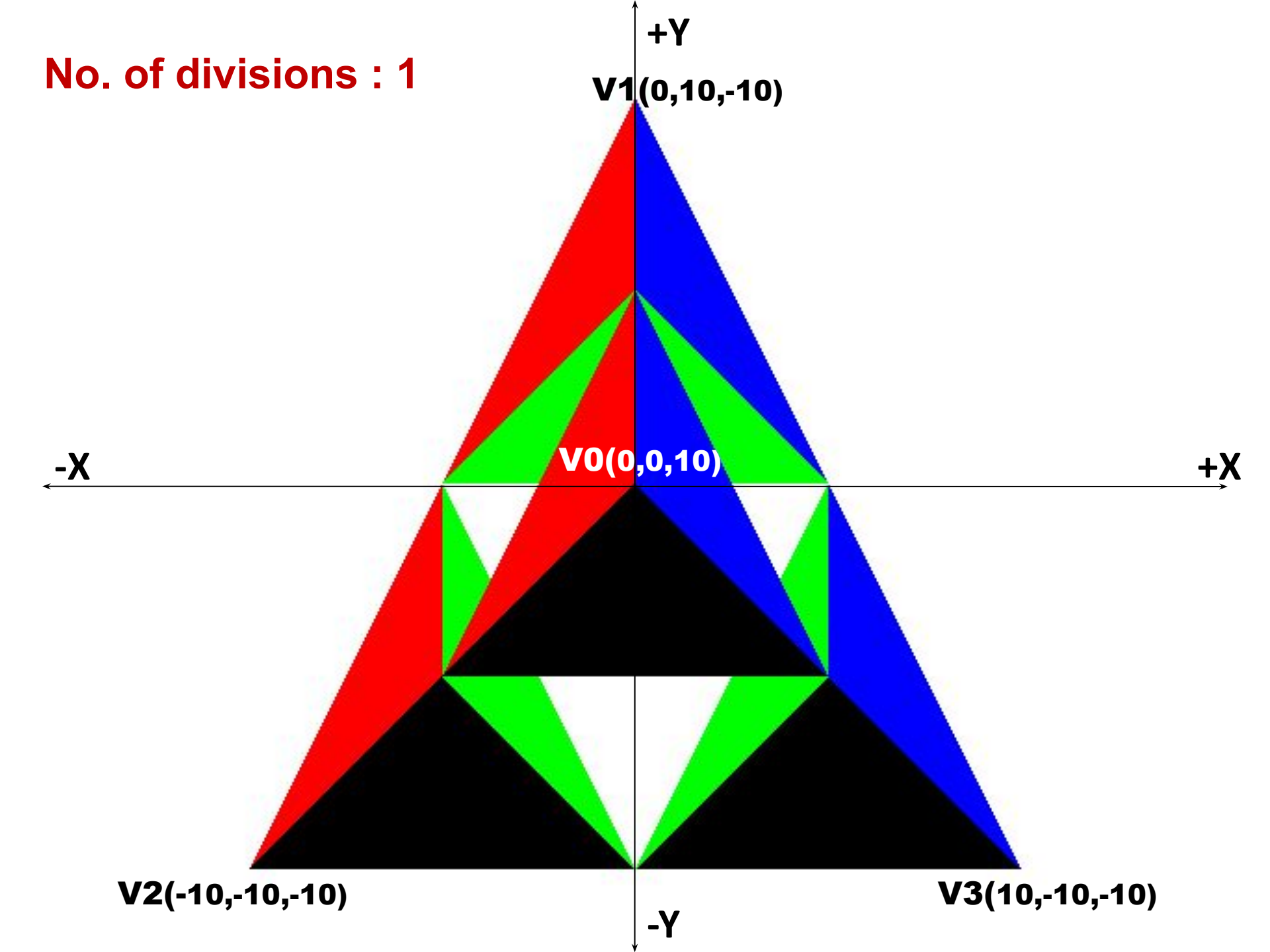
3D Sierpinski gasket

- A fractal which can be constructed by a recursive procedure; at each step a triangle is divided into four new triangles.
- The central triangle is removed and each of the other three treated as the original was, and so on, creating an infinite regression in a finite space.
- **Origin:**
  1970s: named after Wacław Sierpiński (1882–1969), Polish mathematician
- **Application :**
  Fractal geometry

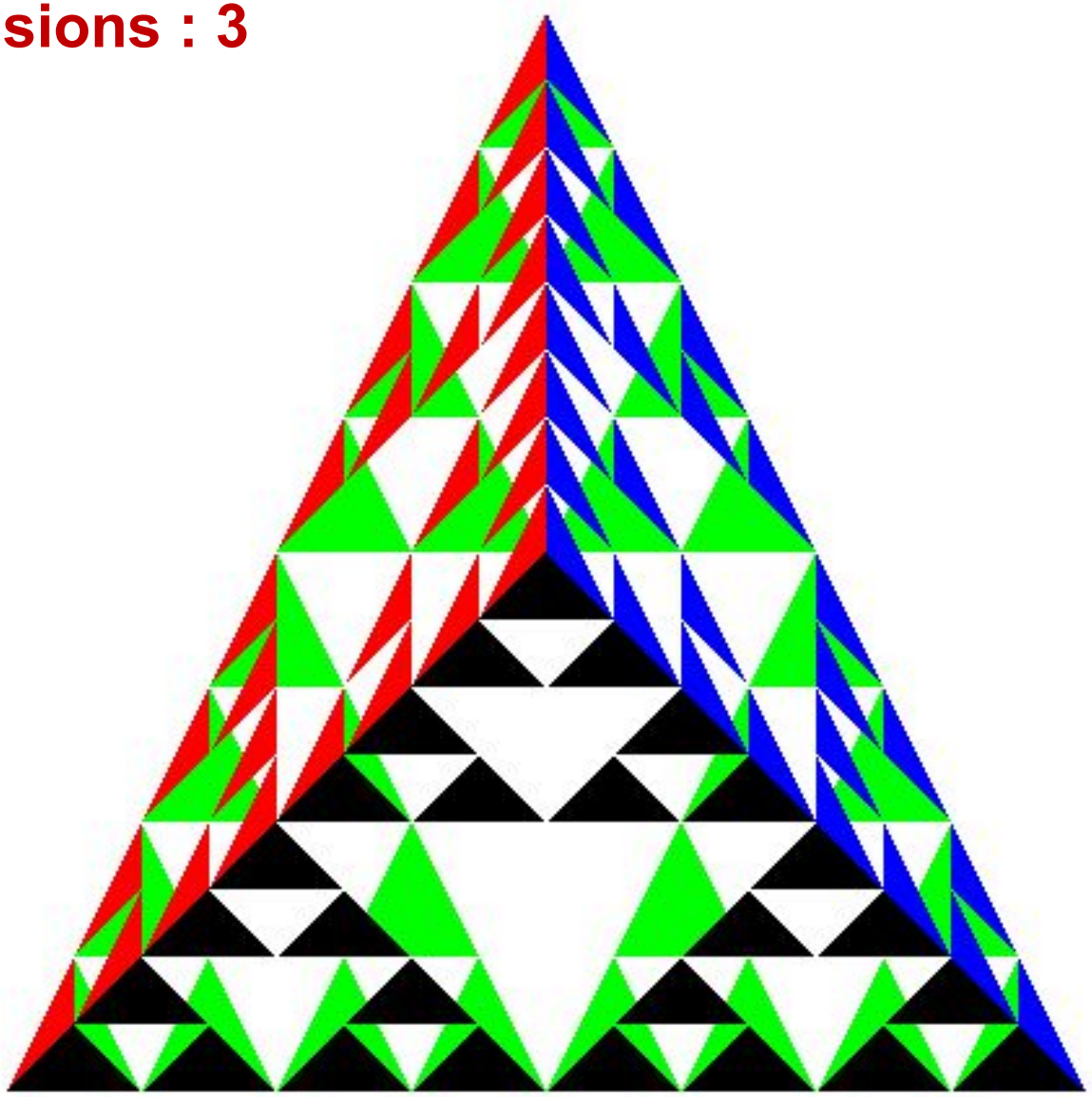


No. of divisions : 1



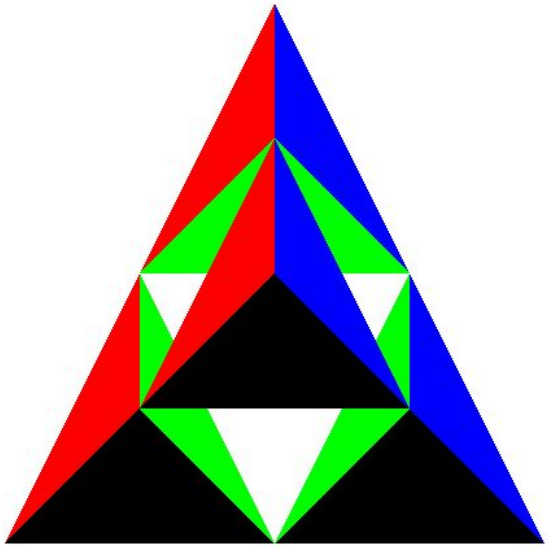

No. of divisions : 1
+Y
V1(0,10,-10)
-X
V0(0,0,10)
+X
V2(-10,-10,-10)
V3(10,-10,-10)
-Y

No. of divisions : 3

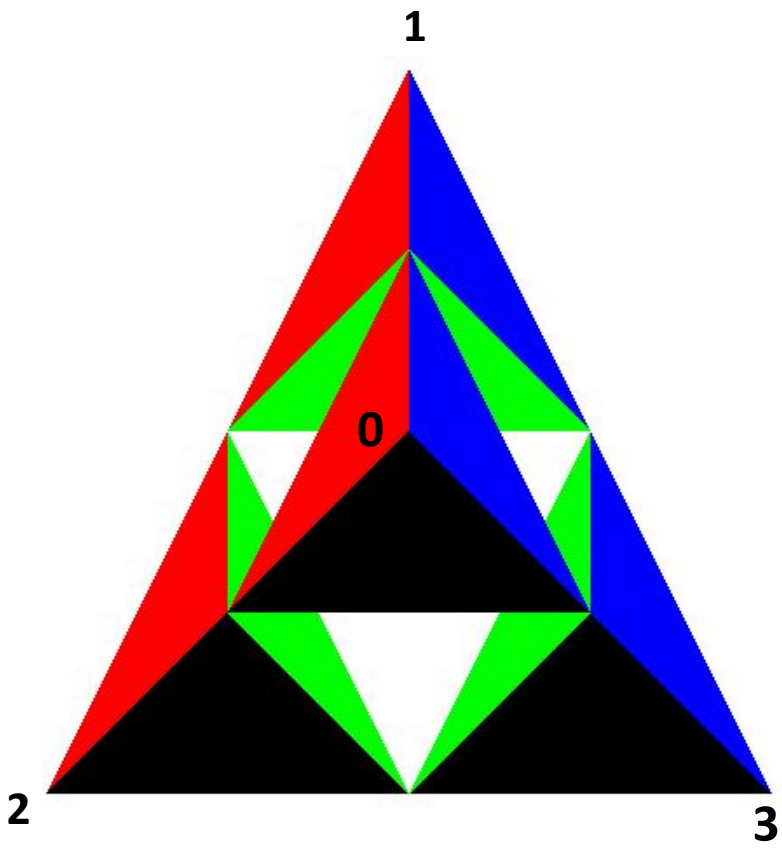


```
#include <stdio.h>
#include <GL/glut.h>

typedef float point[3];

/* initial tetrahedron */
point v[4] =  {    {0.0, 0.0, 10.0},  {0.0, 10.0, -10.0},
            {-10.0, -10.0, -10.0}, {10.0, -10.0, -10.0}
             };
int n;

/* display one triangle */
void triangle( point a, point b, point c)
{     glBegin(GL_POLYGON);
            glVertex3fv(a);
            glVertex3fv(b);
            glVertex3fv(c);
      glEnd();
}
```

```
/* triangle subdivision */
void divide_triangle(point a, point b, point c, int m)
{
        point p1, p2, p3;
        int i;
        if(m>0)
        {
           for(i=0; i<3; i++) p1[i]=(a[i]+b[i])/2;
           for(i=0; i<3; i++) p2[i]=(a[i]+c[i])/2;
           for(i=0; i<3; i++) p3[i]=(b[i]+c[i])/2;

           divide_triangle(a, p1, p2, m-1);
           divide_triangle(c, p2, p3, m-1);
           divide_triangle(b, p3, p1, m-1);
        }
        else
        triangle(a,b,c); /* draw triangle at end of recursion */
}
```



```
/* Apply triangle subdivision to faces of tetrahedron */
void tetrahedron( int m)
{
    glColor3f(1.0,0.0,0.0); //anti clockwise-visible
    divide_triangle(v[0], v[1], v[2], m);
    glColor3f(0.0,1.0,0.0); //clockwise-not visible surface
    divide_triangle(v[3], v[2], v[1], m);
    glColor3f(0.0,0.0,1.0);
     divide_triangle(v[0], v[3], v[1], m);
    glColor3f(0.0,0.0,0.0);
    divide_triangle(v[0], v[2], v[3], m);
}
```



```
void display()
{
   glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
   // to reset the buffer  and for hidden surface removal
   glLoadIdentity(); // loading identity matrix
   tetrahedron(n);
   glFlush();
}

void myReshape(int w, int h) // zoom in and zoom out
{
   glViewport(0, 0, w, h); // to set the viewport according to changed w and h
   glMatrixMode(GL_PROJECTION);
   // matrix mode – defines CTM, GL_PROJECTION  to view in ortho
   glLoadIdentity(); // identity matrix resets the matrix to its default state
   if (w <= h)
   glOrtho(-12.0, 12.0, -12.0 * (GLfloat) h / (GLfloat) w, 12.0 * (GLfloat) h / (GLfloat) w, -12.0, 12.0);
   else
   glOrtho(-12.0 * (GLfloat) w / (GLfloat) h, 12.0 * (GLfloat) w / (GLfloat) h, -12.0, 12.0, -12.0, 12.0);
   glMatrixMode(GL_MODELVIEW); // transformations done in model view
   glutPostRedisplay();
}
```

```
void main(int argc, char **argv)
{
    printf(" No. of Divisions ? ");
    scanf("%d",&n);
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB | GLUT_DEPTH);
    /* This is to view the behind triangle */
    glutInitWindowSize(500, 500);
    glutCreateWindow("3D Gasket");
    glClearColor (1.0, 1.0, 1.0, 1.0);
    glutReshapeFunc(myReshape);
    glutDisplayFunc(display);
    glEnable(GL_DEPTH_TEST);  // enable the hidden surface
    glutMainLoop();
}
```

Input and Interaction: Objectives

- Introduce the basic input devices
  - Physical Devices
  - Logical Devices
  - Input Modes
- Event-driven input
- Programming event input with GLUT

Project Sketchpad

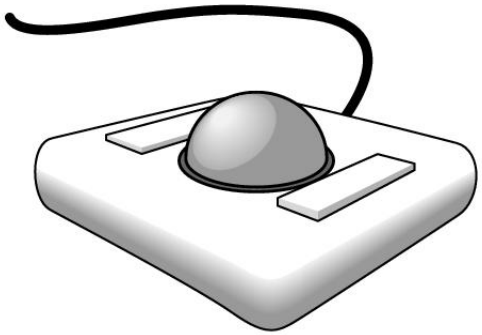
- Ivan Sutherland (MIT 1963) established the basic interactive paradigm that characterizes interactive computer graphics:
  - User sees an *object* on the display
  - User points to (*picks*) the object with an input device (light pen, mouse, trackball)
  - Object changes (moves, rotates, morphs)
  - Repeat

Graphical Input

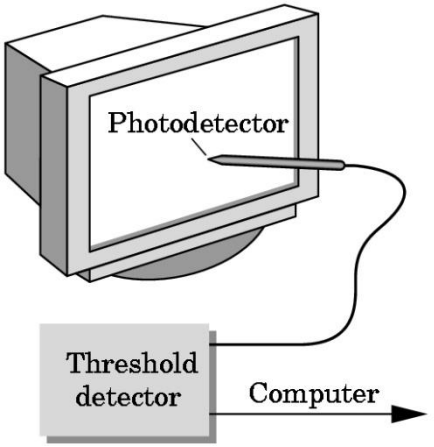
- Devices can be described either by
  - Physical properties
    - Mouse
    - Keyboard
    - Trackball
  - Logical Properties
    - What is returned to program via API
      - **A position**
      - **An object identifier**
- Modes
  - How and when input is obtained
    - Request or event

Physical Devices

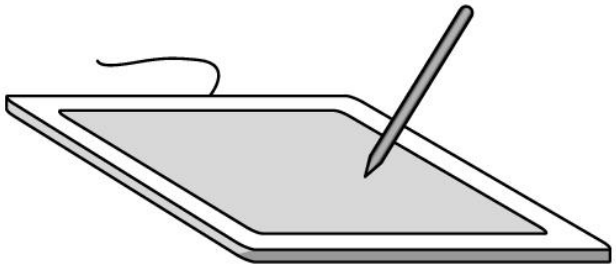
mouse



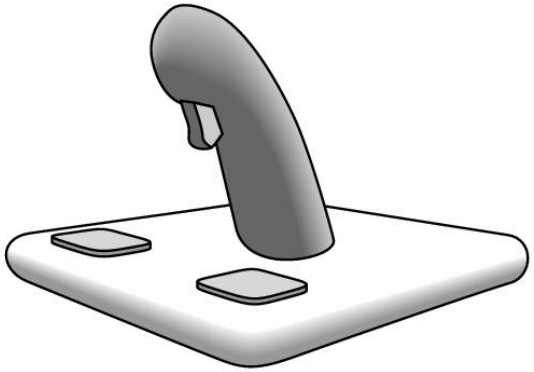
trackball



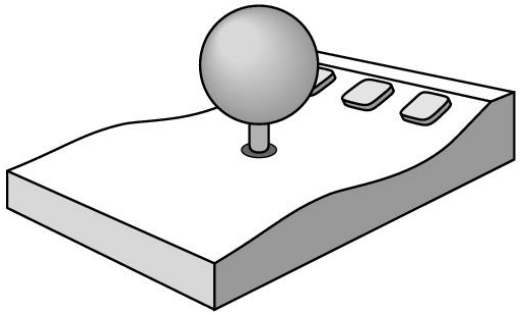
light pen



data tablet



joy stick



space ball

Incremental (Relative) Devices

- Devices such as the data tablet return a position directly to the operating system
- Devices such as the mouse, trackball, and joy stick return incremental inputs (or velocities) to the operating system
  - Must integrate these inputs to obtain an absolute position
    - Rotation of cylinders in mouse
    - Roll of trackball
    - Difficult to obtain absolute position
    - Can get variable sensitivity

Logical Devices

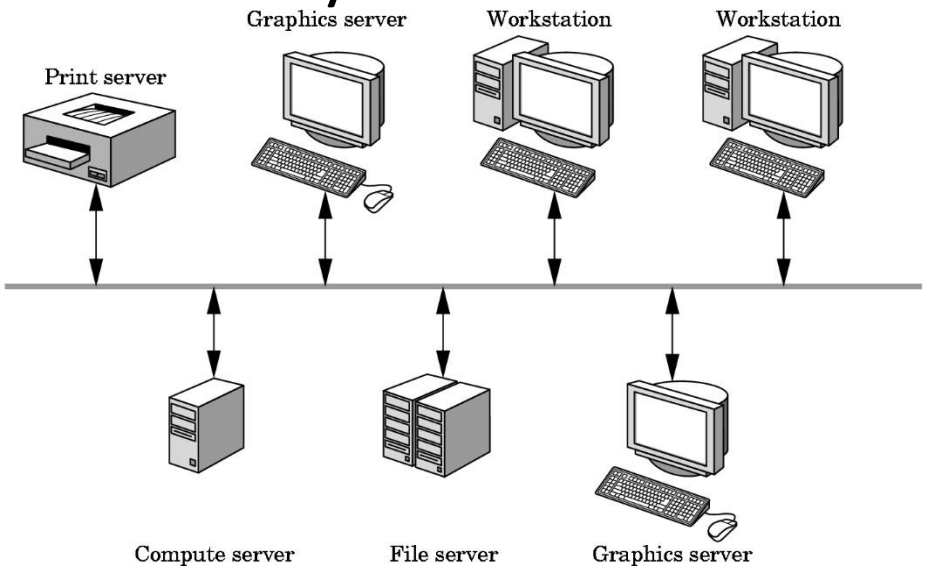
- Consider the C and C++ code
  - C++: **`cin >> x;`**
  - C: **`scanf ("%d", &x);`**
- What is the input device?
  - Can't tell from the code
  - Could be keyboard, file, output from another program
- The code provides *logical input*
  - A number (an **`int`**) is returned to the program regardless of the physical device

Graphical Logical Devices

- Graphical input is more varied than input to standard programs which is usually numbers, characters, or bits
- Two older APIs (GKS, PHIGS) defined six types of logical input
  - **Locator**: return a position
  - **Pick**: return ID of an object
  - **Keyboard**: return strings of characters
  - **Stroke**: return array of positions
  - **Valuator**: return floating point number
  - **Choice**: return one of n items

X Window Input

- The X Window System introduced a client-server model for a network of workstations
  - **Client**: OpenGL program
  - **Graphics Server**: bitmap display with a pointing device and a keyboard

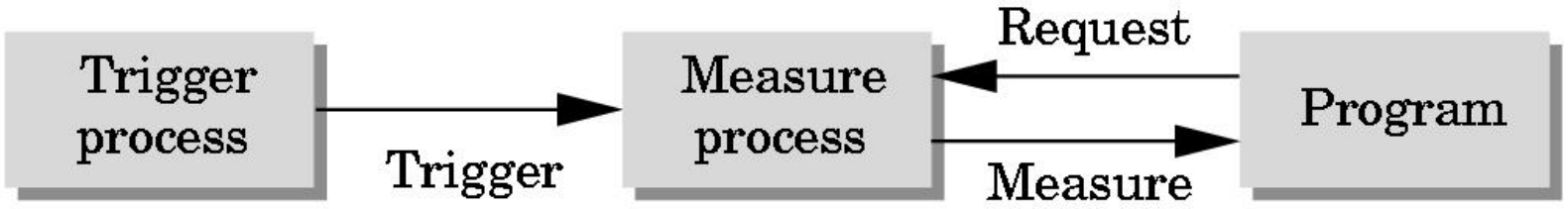


Input Modes

- Input devices contain a *trigger* which can be used to send a signal to the operating system
  - Button on mouse
  - Pressing or releasing a key
- When triggered, input devices return information (their *measure*) to the system
  - Mouse returns position information
  - Keyboard returns ASCII code

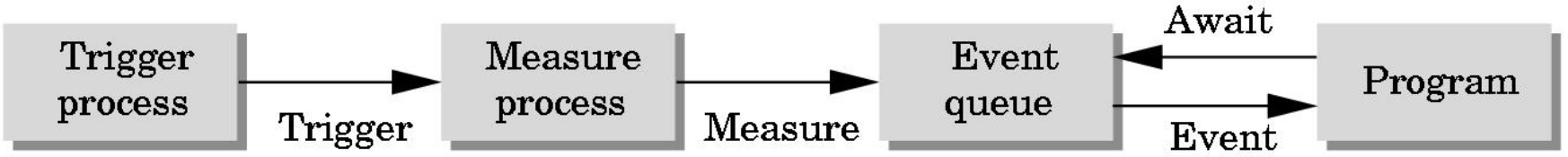
Request Mode

- Input provided to program only when user triggers the device
- Typical of keyboard input
  - Can erase (backspace), edit, correct until enter (return) key (the trigger) is pressed



Event Mode

- Most systems have more than one input device, each of which can be triggered at an arbitrary time by a user
- Each trigger generates an *event* whose measure is put in an *event queue* which can be examined by the user program



Event Types

- Window: resize, expose, iconify
- Mouse: click one or more buttons
- Motion: move mouse
- Keyboard: press or release a key
- Idle: no event
  - Define what should be done if no other event is in queue

Callbacks

- Programming interface for event-driven input
- Define a *callback function* for each type of event the graphics system recognizes
- This user-supplied function is executed when the event occurs
- GLUT example:
  `glutMouseFunc(mymouse)`

mouse callback function

GLUT callbacks

GLUT recognizes a subset of the events recognized by any particular window system (Windows, X, Macintosh)

- **`glutDisplayFunc`**
- **`glutMouseFunc`**
- **`glutReshapeFunc`**
- **`glutKeyboardFunc`**
- **`glutIdleFunc`**
- **`glutMotionFunc`, `glutPassiveMotionFunc`**

GLUT Event Loop

- Recall that the last line in **`main.c`** for a program using GLUT must be

```
glutMainLoop();
```

which puts the program in an infinite event loop

- In each pass through the event loop, GLUT
  - looks at the events in the queue
  - for each event in the queue, GLUT executes the appropriate callback function if one is defined
  - if no callback is defined for the event, the event is ignored

The display callback

- The display callback is executed whenever GLUT determines that the window should be refreshed, for example
  - When the window is first opened
  - When the window is reshaped
  - When a window is exposed
  - When the user program decides it wants to change the display
- In **`main.c`**
  - **`glutDisplayFunc(mydisplay)`** identifies the function to be executed
  - Every GLUT program must have a display callback

Posting redisplays

- Many events may invoke the display callback function
  - Can lead to multiple executions of the display callback on a single pass through the event loop
- We can avoid this problem by instead using

```
glutPostRedisplay();
```

  which sets a flag.
- GLUT checks to see if the flag is set at the end of the event loop
- If set then the display callback function is executed

The mouse callback

```
glutMouseFunc(mymouse)
void mymouse(GLint button, GLint
  state, GLint x, GLint y)
```

- Returns
  - which button (**GLUT_LEFT_BUTTON**, **GLUT_MIDDLE_BUTTON**, **GLUT_RIGHT_BUTTON**) caused event
  - state of that button (**GLUT_UP**, **GLUT_DOWN**)
  - Position in window

Positioning

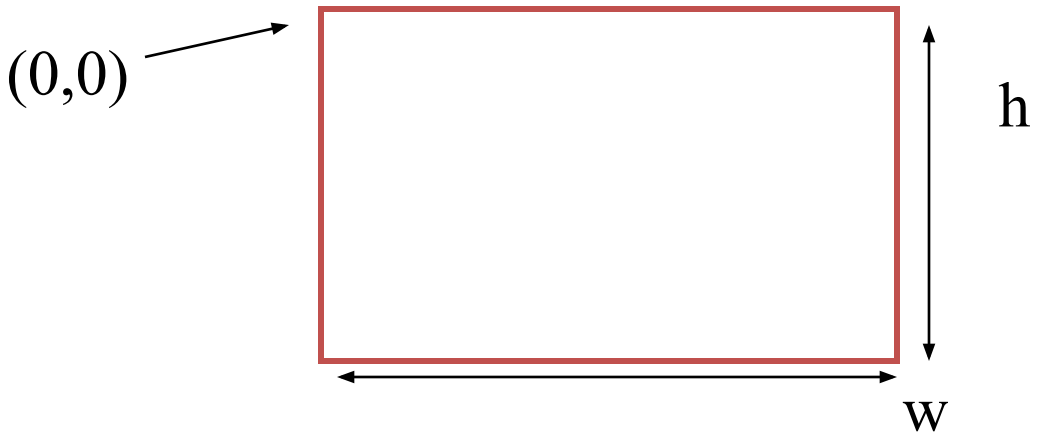
The position in the screen window is usually measured in pixels with the origin at the top-left corner

Consequence of refresh done from top to bottom

OpenGL uses a world coordinate system with origin at the bottom left

Must invert *y* coordinate returned by callback by height of window

y = h – y;



Terminating a program

- We can use the simple mouse callback

```
void mouse(int btn, int state, int x, int y)
{
    if(btn==GLUT_RIGHT_BUTTON && state==GLUT_DOWN)
    exit(0);
}
```

Using the mouse position

- In the next example, we draw a small square at the location of the mouse each time the left mouse button is clicked
- This example does not use the display callback but one is required by GLUT; We can use the empty display callback function

```
mydisplay(){}
```

Drawing squares at cursor location

```
void mymouse(int btn, int state, int x, int y)
{
   if(btn==GLUT_RIGHT_BUTTON && state==GLUT_DOWN)
    exit(0);
     if(btn==GLUT_LEFT_BUTTON && state==GLUT_DOWN)
    drawSquare(x, y);
}
void drawSquare(int x, int y)
{
    y=w-y; /* invert y position */
    glColor3ub( (char) rand()%256, (char) rand )%256,
   (char) rand()%256); /* a random color */
    glBegin(GL_POLYGON);
        glVertex2f(x+size, y+size);
        glVertex2f(x-size, y+size);
        glVertex2f(x-size, y-size);
        glVertex2f(x+size, y-size);
     glEnd();
}
```

Using the motion callback

- We can draw squares (or anything else) continuously as long as a mouse button is pressed by using the motion callback
  - **`glutMotionFunc(drawSquare)`**
- We can draw squares without pressing a button using the passive motion callback
  - **`glutPassiveMotionFunc(drawSquare)`**

Using the keyboard

```
glutKeyboardFunc(mykey)
void mykey(unsigned char key,
           int x, int y)
```

- Returns ASCII code of key pressed and mouse location

```
void mykey()
{
   if(key == 'Q' | key == 'q')
      exit(0);
}
```

Special and Modifier Keys

- GLUT defines the special keys in **`glut.h`**
  - Function key 1: **`GLUT_KEY_F1`**
  - Up arrow key: **`GLUT_KEY_UP`**
    - **`if(key == 'GLUT_KEY_F1'`** ……
- Can also check of one of the modifiers
  - **`GLUT_ACTIVE_SHIFT`**
  - **`GLUT_ACTIVE_CTRL`**
  - **`GLUT_ACTIVE_ALT`**

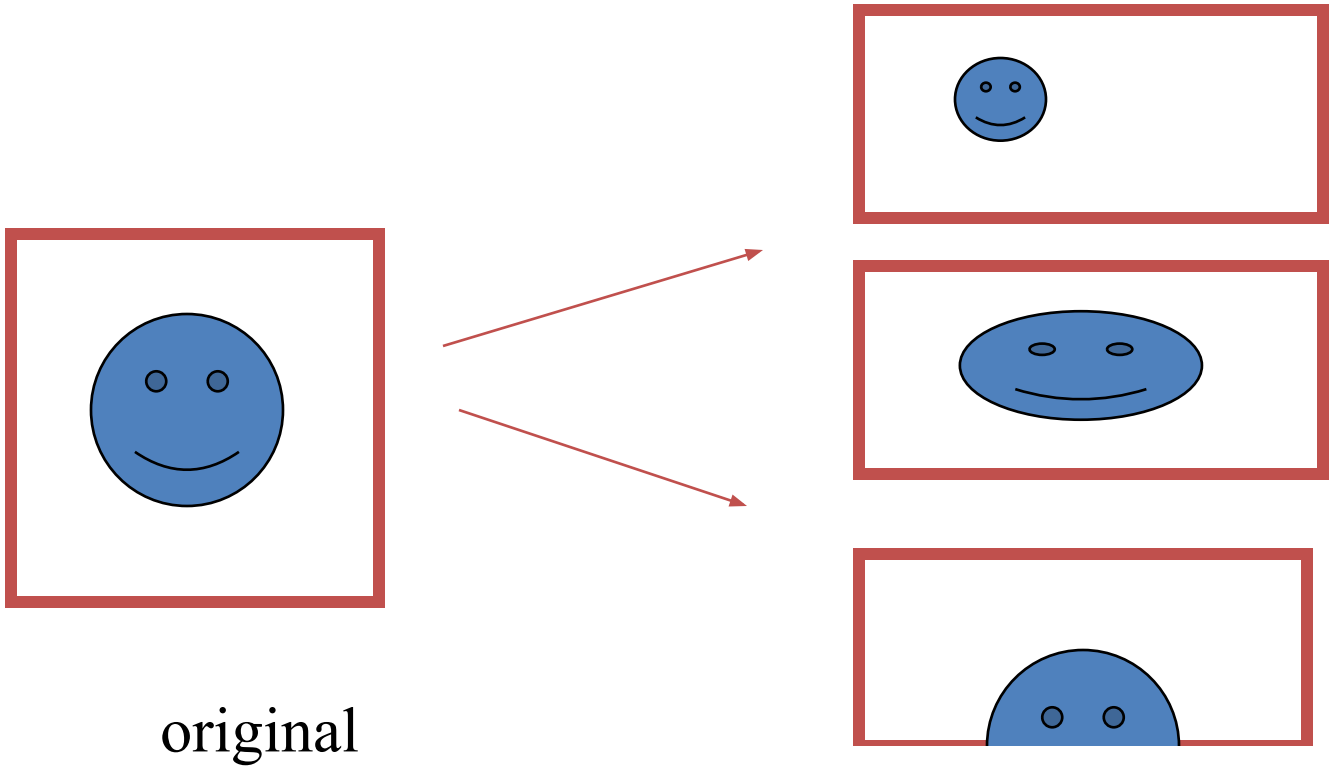
  is pressed by

  `glutGetModifiers()`
  - Allows emulation of three-button mouse with one- or two-button mice

Reshaping the window

- We can reshape and resize the OpenGL display window by pulling the corner of the window
- What happens to the display?
  - Must redraw from application
  - Two possibilities
    - Display part of world
    - Display whole world but force to fit in new window
      - **Can alter aspect ratio**

Reshape possiblities



The Reshape callback

```
glutReshapeFunc(myreshape)
void myreshape( int w, int h)
```

- Returns width and height of new window (in pixels)
- A redisplay is posted automatically at end of execution of the callback
- GLUT has a default reshape callback but you probably want to define your own

- The reshape callback is good place to put viewing functions because it is invoked when the window is first opened

Example Reshape

- This reshape preserves shapes by making the viewport and world window have the same aspect ratio

```
void myReshape(int w, int h)
{
    glViewport(0, 0, w, h);
    glMatrixMode(GL_PROJECTION); /* switch matrix mode */
    glLoadIdentity();
    if (w <= h)
        gluOrtho2D(-2.0, 2.0, -2.0 * (GLfloat) h / (GLfloat)
w,
            2.0 * (GLfloat) h / (GLfloat) w);
    else  gluOrtho2D(-2.0 * (GLfloat) w / (GLfloat) h, 2.0 *
            (GLfloat) w / (GLfloat) h, -2.0, 2.0);
    glMatrixMode(GL_MODELVIEW); /* return to modelview mode */
}
```

Picking

- Identify a user-defined object on the display
- In principle, it should be simple because the mouse gives the position and we should be able to determine to which object(s) a position corresponds
- Practical difficulties
  - Pipeline architecture is feed forward, hard to go from screen back to world
  - Complicated by screen being 2D, world is 3D
  - How close do we have to come to object to say we selected it?

Three Approaches

- Hit list
  - Most general approach but most difficult to implement
- Use back or some other buffer to store object ids as the objects are rendered
- Rectangular maps
  - Easy to implement for many applications

Rendering Modes

- OpenGL can render in one of three modes selected by **`glRenderMode(mode)`**
  - **`GL_RENDER`**: normal rendering to the frame buffer (default)
  - **`GL_FEEDBACK`**: provides list of primitives rendered but no output to the frame buffer
  - **`GL_SELECTION`**: Each primitive in the view volume generates a *hit record* that is placed in a *name stack* which can be examined later

Selection Mode Functions

- **`glSelectBuffer()`**: specifies name buffer
- **`glInitNames()`**: initializes name buffer
- **`glPushName(id)`**: push id on name buffer
- **`glPopName()`**: pop top of name buffer
- **`glLoadName(id)`**: replace top name on buffer

- id is set by application program to identify objects

Display Lists

- Conceptually similar to a graphics file
  - Must define (name, create)
  - Add contents
  - Close
- In client-server environment, display list is placed on server
  - Can be redisplayed without sending primitives over network each time

Display List Functions

- Creating a display list

```
GLuint id;

void init()
{
   id = glGenLists( 1 );
   glNewList( id, GL_COMPILE );
   /* other OpenGL routines */
   glEndList();
}
```

- Call a created list

```
void display()
{
   glCallList( id );
}
```

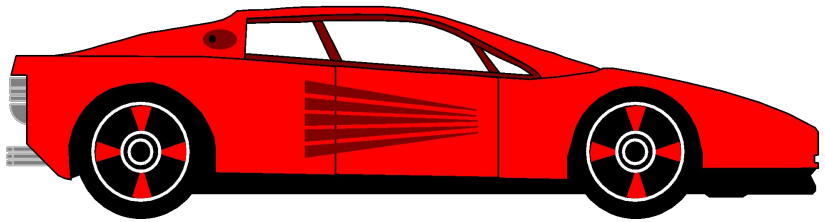
Display Lists and State

- Most OpenGL functions can be put in display lists
- State changes made inside a display list persist after the display list is executed
- Can avoid unexpected results by using **`glPushAttrib`** and **`glPushMatrix`** upon entering a display list and **`glPopAttrib`** and **`glPopMatrix`** before exiting

Hierarchy and Display Lists

- Consider model of a car
  - Create display list for chassis
  - Create display list for wheel

```
glNewList( CAR, GL_COMPILE );
  glCallList( CHASSIS );
  glTranslatef( … );
  glCallList( WHEEL );
  glTranslatef( … );
  glCallList( WHEEL );
    …
glEndList();
```



Animating a Display

- When we redraw the display through the display callback, we usually start by clearing the window
  - **`glClear()`**

then draw the altered display

- Problem: the drawing of information in the frame buffer is decoupled from the display of its contents
  - Graphics systems use dual ported memory
- Hence we can see partially drawn display
  - See the program **`single_double.c`** for an example with a rotating cube

Double Buffering

- Instead of one color buffer, we use two
  - **Front Buffer**: one that is displayed but not written to
  - **Back Buffer**: one that is written to but not displayed
- Program then requests a double buffer in main.c
  - **`glutInitDisplayMode(GL_RGB | GL_DOUBLE)`**
  - At the end of the display callback buffers are swapped

```
void mydisplay()
{
    glClear(GL_COLOR_BUFFER_BIT|….)
.
/* draw graphics here */
.
    glutSwapBuffers()
```

Using the idle callback

- The idle callback is executed whenever there are no events in the event queue
  - **glutIdleFunc(myidle)**
  - Useful for animations

```
void myidle() {
/* change something */
    t += dt
    glutPostRedisplay();
}

Void mydisplay() {
    glClear();
/* draw something that depends on t */
    glutSwapBuffers();
```

Using globals

- The form of all GLUT callbacks is fixed
  - void **mydisplay()**
  - void **mymouse(GLint button, GLint state, GLint x, GLint y)**
- Must use globals to pass information to callbacks

```
float t; /*global */

void mydisplay()
{
/* draw something that depends on t
}
```

Toolkits and Widgets

- Most window systems provide a toolkit or library of functions for building user interfaces that use special types of windows called *widgets*
- Widget sets include tools such as
  - Menus
  - Slidebars
  - Dials
  - Input boxes
- But toolkits tend to be platform dependent
- GLUT provides a few widgets including menus

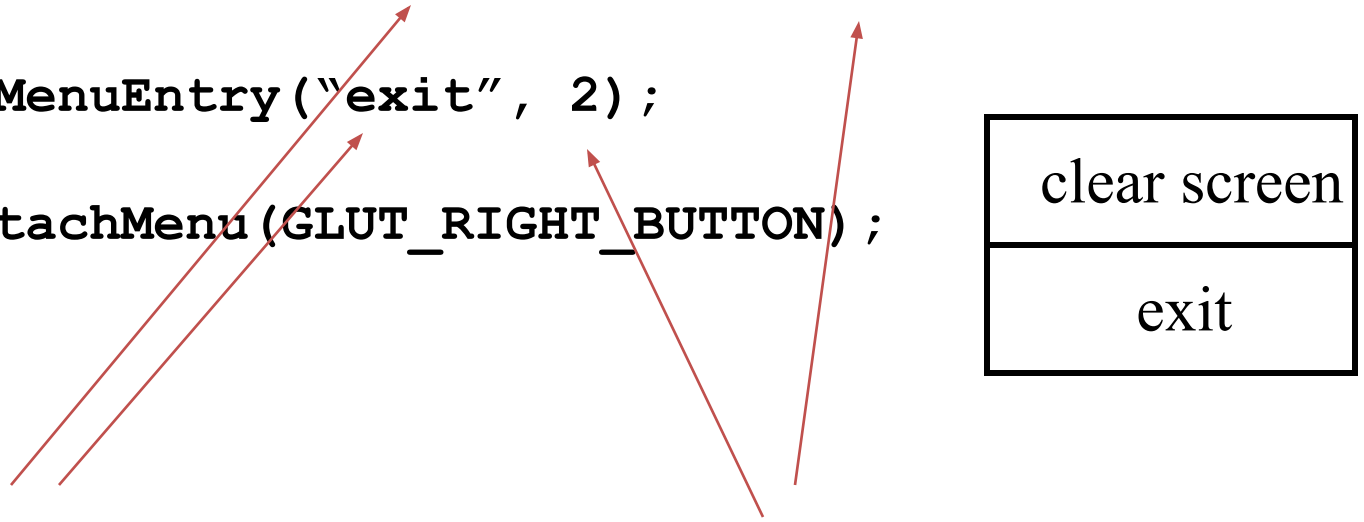
Menus

- GLUT supports pop-up menus
  - A menu can have submenus
- Three steps
  - Define entries for the menu
  - Define action for each menu item
    - Action carried out if entry selected
  - Attach menu to a mouse button

Defining a simple menu

- In **`main.c`**

```
menu_id = glutCreateMenu(mymenu);
glutAddmenuEntry("clear Screen", 1);

gluAddMenuEntry("exit", 2);

glutAttachMenu(GLUT_RIGHT_BUTTON);
```



Menu actions

- Menu callback

```
void mymenu(int id)
{
    if(id == 1) glClear();
    if(id == 2) exit(0);
}
```

- Note each menu has an id that is returned when it is created
- Add submenus by

```
glutAddSubMenu(char *submenu_name, submenu id)
```

entry in parent menu

Other functions in GLUT

- Dynamic Windows
  - Create and destroy during execution
- Subwindows
- Multiple Windows
- Changing callbacks during execution
- Timers
- Portable fonts
  - **`glutBitmapCharacter`**
  - **`glutStrokeCharacter`**